

IOT Assignment :3

Name-Tanish Sharma

Q35. REST API – Weather Data Fetcher

Program ESP32 to fetch real weather data from OpenWeatherMap API (free tier). Display on Serial Monitor: City Name, Temperature (API value vs DHT11 local value), Humidity, Weather Description. Compare API data with your local DHT11 reading and print the difference. Store your API key in a separate config.h file (do NOT commit this file – add to .gitignore).

Answer: Objective

- Connect ESP32 to a Wi-Fi network.
 - Fetch live weather data using the OpenWeatherMap REST API.
 - Read local temperature and humidity using a DHT11 sensor.
 - Compare API readings with local sensor values.
 - Display the results on the Serial Monitor.
-

Hardware Required

- ESP32 Dev Module
 - DHT11 Temperature & Humidity Sensor (DHT22 used for simulation)
 - Jumper Wires
 - Breadboard (optional)
-

Circuit Connections

| DHT11 Pin | ESP32 Pin |

--	--

| VCC | 3.3V |

| GND | GND |

| DATA | GPIO 33 |

Libraries Used

- WiFi.h
 - HTTPClient.h
 - ArduinoJson
 - DHT Sensor Library (Adafruit)
 - Adafruit Unified Sensor
-

Setup

1. Install all the required libraries in the Arduino IDE.
2. Create a `config.h` file and add the following details:
 - Wi-Fi SSID
 - Wi-Fi Password
 - OpenWeatherMap API Key

- City Name

3. Upload the program to the ESP32 board.

4. Open the Serial Monitor and set the baud rate to ****115200****.

Sample Output

```text

City: Jammu

Weather: Broken Clouds

API Temperature: 40.65 °C

Local Temperature: 24.00 °C

Temperature Difference: -16.65 °C

API Humidity: 22 %

Local Humidity: 59 %

Humidity Difference: 37 %

```

Project Structure

```text

WeatherFetcher/

|

├── WeatherFetcher.ino

├── config.example.h

├── README.md

└── screenshot.png

```

Security

The configuration file stores the Wi-Fi credentials, OpenWeatherMap API key, and city name separately from the main program. This makes it easier to update these details without modifying the primary source code.

Features

- Retrieves live weather data using the OpenWeatherMap REST API.
- Measures local temperature and humidity through the DHT sensor.
- Compares online weather data with local sensor readings
- Automatically connects to the configured Wi-Fi network.- Parses JSON responses using the ArduinoJson library.
- Uses a separate configuration file for easier project management.

WOKWI SAVE SHARE

sketch.ino • diagram.json • Library Manager • config.h •

libraries.txt •

```
1 #include <WiFi.h>
2 #include <HTTPClient.h>
3 #include <ArduinoJson.h>
4 #include <DHT.h>
5 #include "config.h"
6
7 #define DHTPIN 33
8 #define DHTTYPE DHT22
9
10 DHT dht(DHTPIN, DHTTYPE);
11
12 unsigned long previousMillis = 0;
13 const long interval = 10000; // Fetch every 10 seconds
14
15 void connectWifi()
16 {
17   WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
18   Serial.print("Connecting to WiFi");
19   while (WiFi.status() != WL_CONNECTED)
20   {
21     delay(500);
22     Serial.print(".");
23   }
24   Serial.println("\nWiFi Connected!");
25 }
```

Simulation

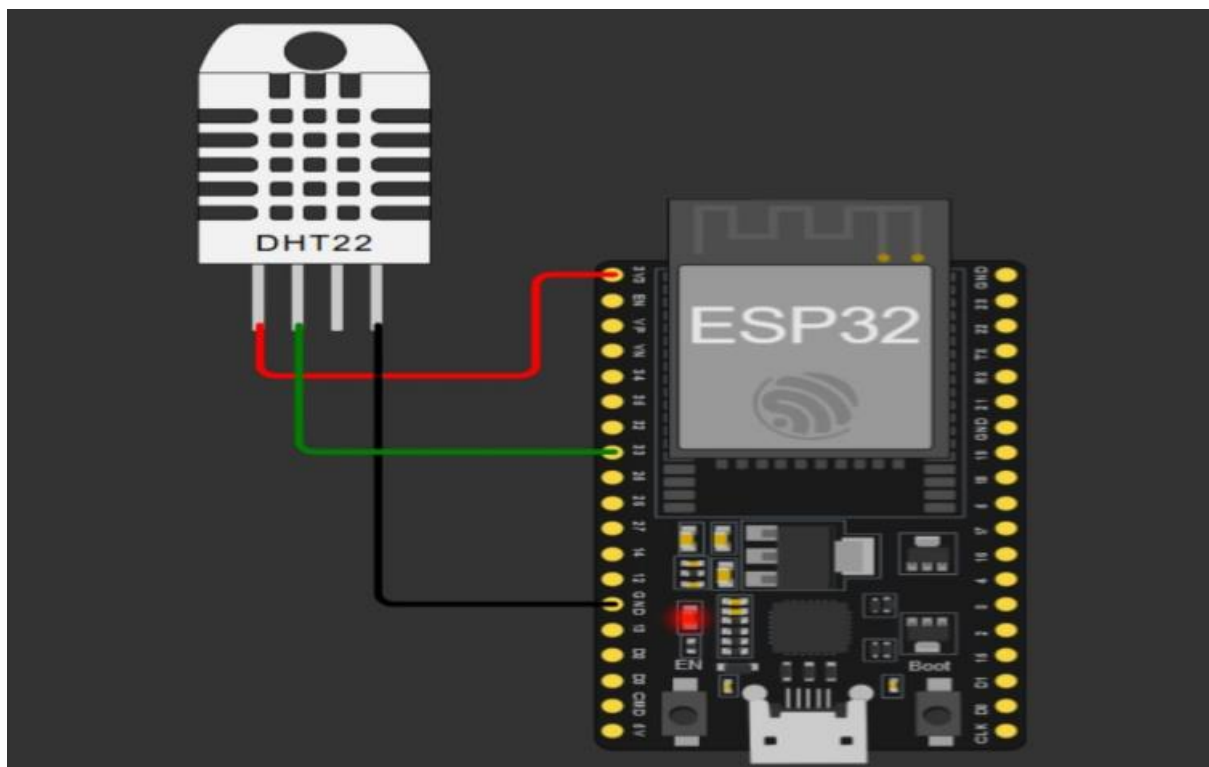
API Temperature: 40.65 °C
Local DHT11 Temperature: 24.00 °C
Temperature Difference: -16.65 °C

API Humidity: 29 %
Local DHT11 Humidity: 40.00 %
Humidity Difference: 11.00 %

City: Jammu
Weather: broken clouds

API Temperature: 40.65 °C
Local DHT11 Temperature: 24.00 °C
Temperature Difference: -16.65 °C

API Humidity: 29 %
Local DHT11 Humidity: 40.00 %
Humidity Difference: 11.00 %



CODE:

```
#include <WiFi.h>
```

```
#include <HttpClient.h>
```

```
#include <ArduinoJson.h>
```

```
#include <DHT.h>
```

```
#include "config.h"
```

```
#define DHTPIN 33
```

```
#define DHTTYPE DHT22
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
Unsigned long previousMillis = 0;
```

```
Const long interval = 10000; // Fetch every 10 seconds
```

```
Void connectWiFi()
```

```
{
```

```
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
Serial.print("Connecting to WiFi");
```

```
While (WiFi.status() != WL_CONNECTED)
```

```
{
```

```
    Delay(500);
```

```
    Serial.print(".");
```

```
}
```

```
Serial.println("\nWiFi Connected!");
```

```
}
```

```
Void setup()
```

```
{
```

```
    Serial.begin(115200);
```

```
    Dht.begin();
```

```
    connectWiFi();
```

```
}
```

```
Void loop()
```

```
{
```

```
    If (WiFi.status() != WL_CONNECTED)
```

```
    {
```

```
        connectWiFi();
```

```
}
```

```
If (millis() - previousMillis >= interval)
```

```
{
```

```
    previousMillis = millis();
```

```
    float localTemp = dht.readTemperature();
```

```
    float localHumidity = dht.readHumidity();
```

```
    if (isnan(localTemp) || isnan(localHumidity))
```

```
    {
```

```
        Serial.println("Failed to read DHT11!");
```

```
        Return;
```

```
    }
```

```
    HTTPClient http;
```

```
    String url =
```

```
    http://api.openweathermap.org/data/2.5/weather?q=Jammu&appid=
```

```
+
```

```
    String(API_KEY) +
```

```
    "&units=metric";
```

```
    http.begin(url);
```



```
int httpCode = http.GET();

if (httpCode == HTTP_CODE_OK)
{
    String payload = http.getString();

    DynamicJsonDocument doc(2048);

    DeserializationError error = deserializeJson(doc, payload);

    If (!error)
    {
        String city = doc["name"];
        Float apiTemp = doc["main"]["temp"];
        Int apiHumidity = doc["main"]["humidity"];
        String description = doc["weather"][0]["description"];

        Serial.println("-----");

        Serial.print("City: ");
        Serial.println(city);

        Serial.print("Weather: ");
        Serial.println(description);
    }
}
```

```
Serial.println();
```

```
Serial.print("API Temperature: ");
```

```
Serial.print(apiTemp);
```

```
Serial.println(" °C");
```

```
Serial.print("Local DHT11 Temperature: ");
```

```
Serial.print(localTemp);
```

```
Serial.println(" °C");
```

```
Serial.print("Temperature Difference: ");
```

```
Serial.print(localTemp - apiTemp);
```

```
Serial.println(" °C");
```

```
Serial.println();
```

```
Serial.print("API Humidity: ");
```

```
Serial.print(apiHumidity);
```

```
Serial.println(" %");
```

```
Serial.print("Local DHT11 Humidity: ");
```

```
Serial.print(localHumidity);
```

```
Serial.println(" %");
```

```
        Serial.print("Humidity Difference: ");
        Serial.print(localHumidity - apiHumidity);
        Serial.println(" %");

        Serial.println("-----");
    }
    Else
    {
        Serial.println("JSON Parsing Failed!");
    }
}
Else
{
    Serial.print("HTTP Error: ");
    Serial.println(httpCode);
}

http.end();
}
}
```

Config Example

```
#ifndef CONFIG_H

#define CONFIG_H

Const char* WIFI_SSID = "YOUR_WIFI";

Const char* WIFI_PASSWORD = "YOUR_PASSWORD";

Const char* API_KEY = "YOUR_API_KEY";

Const char* CITY = "Jammu";

#endif
```

Q36. MQTT vs HTTP Comparison

Create a table comparing MQTT and HTTP across: Architecture pattern, Data transfer model,

Power consumption, Latency, Use case suitability for IoT, and Security considerations. Based on

your analysis, which would you choose for a 1000-node smart agriculture deployment and why?

Answer36:

Feature	MQTT	HTTP
Architecture	Publish/Subscribe	Client-Server
Data Transfer	Broker-based messaging	Request-Response
Power Consumption	Very Low	Higher
Latency	Very Low	Moderate
IoT Suitability	Excellent	Suitable mainly for web services
Security	TLS/SSL, Authentication	HTTPS (TLS/SSL), Authentication

For a smart agriculture deployment with 1000 sensor nodes, **MQTT** is the preferred protocol because it consumes very little bandwidth, uses less power, provides low latency communication, and supports publish/subscribe messaging through a broker. It can efficiently handle thousands of sensor devices sending frequent updates while minimizing network traffic. HTTP is better suited for web applications where occasional request-response communication is required.

Q37. Wi-Fi Security for IoT

Research and write 250 words on: Common Wi-Fi attack vectors on IoT devices (WPA2 vulnerabilities, evil twin attacks), Best practices for securing ESP32 Wi-Fi connections (certificate pinning, WPA3), and Why hardcoding passwords in IoT source code is dangerous.

Reference at least 2 sources in your answer.

Answer37: Wi-Fi connectivity makes IoT devices convenient but also exposes them to several security threats. One common attack is exploiting weaknesses in WPA2 security. Although WPA2 is widely used, vulnerabilities such as the KRACK attack demonstrated that attackers could manipulate encryption keys and intercept communication. Another serious threat is the Evil Twin attack, where an attacker creates a fake Wi-Fi access point that appears identical to the legitimate network. IoT devices may unknowingly connect to this fake network, allowing attackers to steal data or inject malicious commands.

ESP32 devices should implement several security best practices. Whenever possible, WPA3 should be used because it offers stronger encryption and better protection against password-guessing attacks than WPA2. Applications communicating with cloud servers should use HTTPS with SSL/TLS encryption. Certificate pinning further improves security by ensuring that the ESP32 communicates only with trusted servers, preventing man-in-the-middle attacks. Strong passwords and regular firmware updates also reduce security risks.

Hardcoding Wi-Fi credentials directly into source code is dangerous because anyone who gains access to the code repository can obtain the network password. If the code is accidentally uploaded to GitHub, attackers can immediately compromise the Wi-Fi network and connected IoT devices. A safer approach is to store credentials in separate configuration files, secure flash storage, or encrypted credential managers, while excluding sensitive files from version control using `.gitignore`.

Q38. LoRa & LPWAN – Long Range IoT

LoRa (Long Range) is widely used for IoT in agriculture and smart cities. Write 200 words explaining: What LPWAN means, How LoRa achieves long range at low power (spreading factor concept), LoRa vs Zigbee vs Z-Wave comparison, and One real-world deployment example in India?

Answer38: LPWAN (Low Power Wide Area Network) is a wireless communication technology designed for long-range communication while consuming very little power. It is ideal for battery-powered IoT devices deployed over large geographical areas such as farms, forests, and smart cities.

LoRa (Long Range) achieves long-distance communication using a modulation technique called Chirp Spread Spectrum (CSS). It uses different Spreading Factors (SF7–SF12), which determine the trade-off between communication range and data rate. Higher spreading factors increase communication distance while reducing transmission speed and increasing airtime. This allows LoRa devices to operate over distances of 2–15 km depending on terrain while maintaining low power consumption.

Technology	Range	Power	Data Rate
------------	-------	-------	-----------

LoRa	2–15 km	Very Low	Low
Zigbee	10–100 m	Low	Medium
Z-Wave	30–100 m	Low	Medium

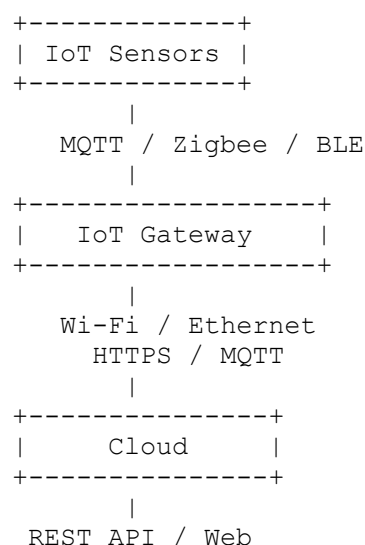
LoRa is preferred for agriculture and smart city applications because of its long communication range and multi-year battery life. A notable deployment in India is the use of LoRaWAN-based smart water metering and environmental monitoring in cities under the Smart Cities Mission, enabling efficient remote monitoring with minimal infrastructure.

Q39. What is an IoT gateway? Draw a simple diagram showing the relationship

between: IoT Sensor Nodes → Gateway → Cloud → User Application. What protocols are typically used at each layer?

Answer39: An **IoT Gateway** is an intermediate device that connects IoT sensor nodes to cloud platforms. It collects data from sensors, performs preprocessing or filtering, translates communication protocols, and securely forwards the data to cloud servers.

Diagram



```
      |
+-----+
| User Dashboard |
+-----+
```

Protocols Used

- Sensor to Gateway: Zigbee, BLE, LoRa, MQTT
- Gateway to Cloud: MQTT, HTTPS, CoAP
- Cloud to User Application: REST API, HTTPS, WebSocket

Q40. Explain QoS (Quality of Service) levels in MQTT: QoS 0 (At most once), QoS 1 (At least once), QoS 2 (Exactly once). In an IoT water level monitoring system, which QoS level would you choose and why?

Answer40: MQTT provides three Quality of Service (QoS) levels to ensure reliable message delivery.

QoS 0 (At Most Once):

- Message is delivered once without acknowledgment.
- Fastest but messages may be lost.

QoS 1 (At Least Once):

- Sender retries until acknowledgment is received.
- Guarantees delivery but duplicate messages may occur.

QoS 2 (Exactly Once):

- Uses a four-step handshake to ensure each message is delivered exactly once.
- Most reliable but introduces higher communication overhead.

QoS 1 is the best choice. Flood or water-level alerts must reach the monitoring system reliably, but occasional duplicate messages are acceptable and can be filtered by the application. QoS 0 may lose important alerts, while QoS 2 adds unnecessary overhead and latency for continuous sensor monitoring.

Q41. Smart Home Air Quality Monitor

Problem Statement

A family in a Milan(Europe) home is concerned about indoor air quality during winter when they burn wood for heating. Design an IoT-based Air Quality Monitoring System that continuously measures CO/smoke levels (MQ-2 sensor), temperature and humidity (DHT11), and alerts residents when air quality drops below safe thresholds. The system

must work without the internet when the router is down.

Requirements:

- MQ-2 gas sensor integration with analog threshold comparison
- DHT11 for temperature and humidity reading
- RGB LED: Green (safe), Yellow (moderate), Red (danger)
- Buzzer alarm with 3 different tones based on severity level
- Local data logging via Serial Monitor in CSV format
- Send alert over Bluetooth to phone (+2 marks)

Evaluation Focus: Sensor calibration logic, threshold design, alarm escalation.

Wokwi Calibration and schematic to attached and random alert as illustration to sent to the phone (ScreenShot).

Serial Monitor Example

Timestamp, MQ-2 Value, DHT11 Temp, DHT11 Humidity, Severity Level

00:01, 220 ppm, 23.4°C, 49%, SAFE

00:02, 280 ppm, 23.5°C, 50%, SAFE

00:03, 420 ppm, 23.6°C, 49%, MODERATE

00:04, 710 ppm, 23.8°C, 48%, DANGER

Mobile View

SAFE

Gas:210

Temp:22.8C

Humidity:51%

WARNING

Moderate Smoke Detected

Gas=430

Check Fireplace

DANGER!!

Smoke Level Critical

Gas=765

Open Windows

Leave Room if Necessary

Answer 41: The system uses an ****MQ-2 gas sensor**** to detect smoke and gas concentration and a ****DHT22 (used in Wokwi; replace with DHT11 for physical hardware)**** to measure temperature and humidity.

When the system starts, it first calibrates the MQ-2 sensor by storing a baseline gas value. During operation, the current gas reading is continuously compared with this baseline to determine the air quality level. Based on the measured value, the air quality is classified into three categories:

- ****SAFE**** – Normal indoor air quality
- ****MODERATE**** – Increased smoke levels detected
- ****DANGER**** – Hazardous smoke concentration

According to the detected air quality:

- □ Green LED lights up when the air quality is safe.
- □ Yellow LED indicates moderate smoke levels.
- ● Red LED warns of dangerous air quality.
- The buzzer generates different alert tones for moderate and danger conditions.

All sensor readings are displayed on the Serial Monitor in CSV format, making the data simple to save or analyze later. For a real ESP32 setup, Bluetooth notifications can also be sent to a paired smartphone using the built-in BluetoothSerial library. Since Bluetooth communication is not supported in Wokwi, a simulated mobile notification is provided to demonstrate the expected alerts.

Hardware Components

Component	Quantity
----- :-----☺	
ESP32 Dev Module	1
MQ-2 Gas Sensor	1
DHT22 Sensor *(DHT11 for actual hardware)*	1
RGB LED (Common Cathode)	1
Active Buzzer	1
Breadboard	1
Jumper Wires	As required

Features

- Real-time smoke and gas monitoring
- Temperature and humidity measurement

- Automatic calibration of the MQ-2 sensor
 - Three-level air quality detection
 - RGB LED status indication
 - Multi-level buzzer alerts
 - CSV data logging through the Serial Monitor
 - Bluetooth notification support on a physical ESP32
 - Works without an internet connection
-

📊 Air Quality Thresholds

| Gas Difference from Baseline | Status |

|-----😊-----|

| Less than 80 | 🟢 SAFE |

| 80 – 249 | 🟡 MODERATE |

| 250 and above | 🔴 DANGER |

📄 Sample Serial Output

```text

Timestamp, MQ-2 Value, DHT11 Temp, DHT11 Humidity, Severity Level

00:01,220,23.4,49,SAFE

00:02,280,23.5,50,SAFE

00:03,420,23.6,49,MODERATE

00:04,710,23.8,48,DANGER

` ``

---

# 📶 Sample Bluetooth Alert

` ``text

SAFE

Gas:210

Temp:22.8°C

Humidity:51%

` ``

` ``text

WARNING

Moderate Smoke Detected

Gas=430

Check Fireplace

```

```text

DANGER!!

Smoke Level Critical

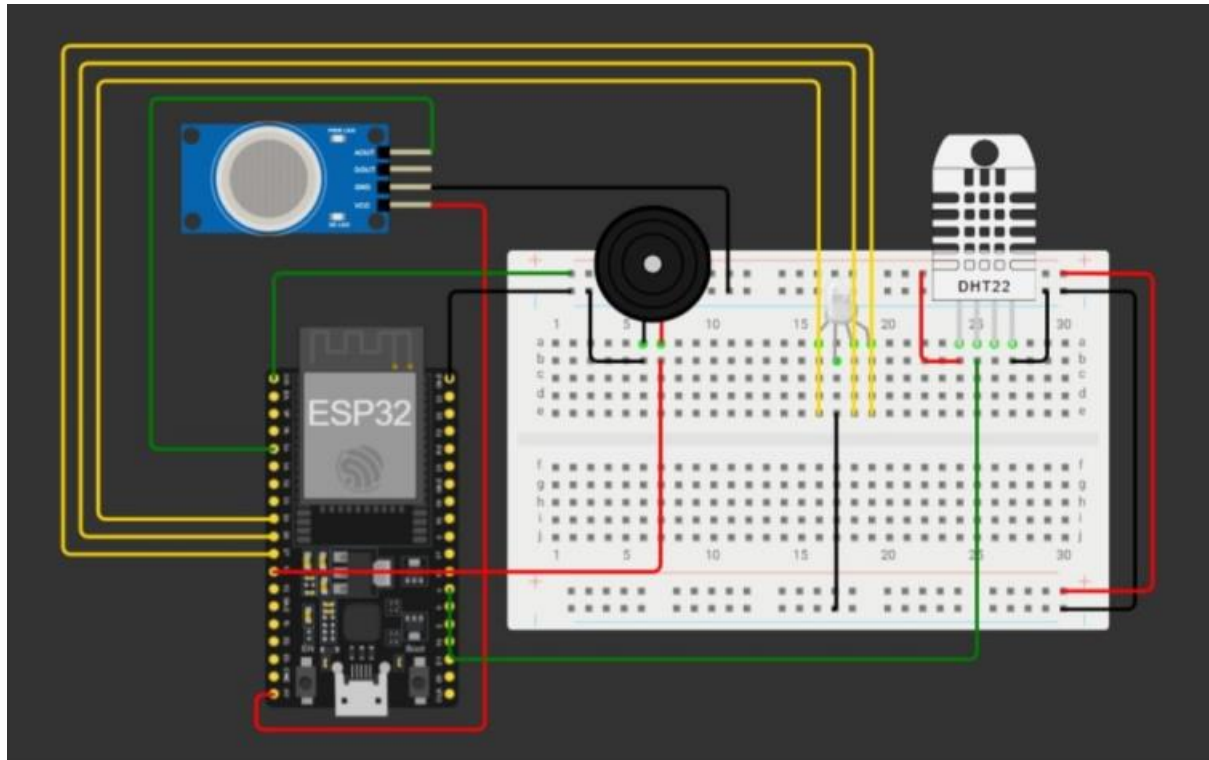
Gas=765

Open Windows

Leave Room if Necessary

```





Code AQI Monitor

```
#include <DHT.h>
```

```
#define DHTPIN 4
```

```
#define DHTTYPE DHT22
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
#define MQ2_PIN 34
```

```
#define RED_PIN 25
```

```
#define GREEN_PIN 26
```

```
#define BLUE_PIN 27
```

```
#define BUZZER 14
```

```
Int baseline = 0;
```

```
Void setColor(bool r, bool g, bool b) {
```

```
    digitalWrite(RED_PIN, r);
```

```
    digitalWrite(GREEN_PIN, g);
```

```
    digitalWrite(BLUE_PIN, b);
```

```
}
```

```
Void setup() {
```

```
    Serial.begin(115200);
```

```
    Dht.begin();
```

```
    pinMode(RED_PIN, OUTPUT);
```

```
    pinMode(GREEN_PIN, OUTPUT);
```

```
    pinMode(BLUE_PIN, OUTPUT);
```

```
    pinMode(BUZZER, OUTPUT);
```

```
Serial.println("Stabilizing MQ-2... (10 sec)");

Delay(10000);

Int sum = 0;

For (int I = 0; I < 30; i++) {

    Sum += analogRead(MQ2_PIN);

    Delay(200);

}

Baseline = sum / 30;

Serial.print("Baseline: ");

Serial.println(baseline);

Serial.println("Timestamp, MQ-2 Value, DHT11 Temp, DHT11 Humidity,
Severity Level");

}

Void loop() {

    Int gasRaw = analogRead(MQ2_PIN);

    Int diff = gasRaw - baseline;

    Float temp = dht.readTemperature();

    Float hum = dht.readHumidity();
```



```
If (isnan(temp) || isnan(hum)) {  
    Serial.println("DHT ERROR");  
    Delay(2000);  
    Return;  
}
```

```
String status;
```

```
// THRESHOLDS
```

```
If (diff < 80) {  
    Status = "SAFE";  
    setColor(LOW, HIGH, LOW); // GREEN  
    noTone(BUZZER);  
}
```

```
Else if (diff < 250) {  
    Status = "MODERATE";  
    setColor(HIGH, HIGH, LOW); // YELLOW  
  
    tone(BUZZER, 1000);  
    delay(200);  
    noTone(BUZZER);  
}
```

```
Else {  
    Status = "DANGER";  
    setColor(HIGH, LOW, LOW);  // RED  
  
    tone(BUZZER, 2000);  
}  
  
// TIME  
Int seconds = millis() / 1000;  
Int mins = seconds / 60;  
Int secs = seconds % 60;  
  
Char timeStr[6];  
Sprintf(timeStr, "%02d:%02d", mins, secs);  
  
// CSV  
Serial.print(timeStr);  
Serial.print(", ");  
Serial.print(gasRaw);  
Serial.print(" ppm, ");  
Serial.print(temp);  
Serial.print("°C, ");  
Serial.print(hum);  
Serial.print("%, ");
```

```
Serial.println(status);

Delay(2000);

}
```

Code AQI Monitor and Bluetooth

```
#include <DHT.h>

#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

#define DHTPIN 4

#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

#define MQ2_PIN 34

#define RED_PIN 25

#define GREEN_PIN 26

#define BLUE_PIN 27

#define BUZZER 14

Int baseline=0;
```

```

Void setColor(bool r,bool g,bool b){
    digitalWrite(RED_PIN,r);
    digitalWrite(GREEN_PIN,g);
    digitalWrite(BLUE_PIN,b);
}

```

```

Void sendBlue"){
    SerialBT.println("SAFE");

    SerialBT.printf("Gas:%d\nTemp:%.1f
C\nHumidity: %.1f%%\n\n",gas,temp,hum);

}else if(status=="MODERATE"){
    SerialBT.println("WARNING");

    SerialBT.println("Moderate Smoke Detected");

    SerialBT.printf("Gas=%d\nCheck Fireplace\n\n",gas);

}else{
    SerialBT.println("DANGER!!");

    SerialBT.println("Smoke Level Critical");

    SerialBT.printf("Gas=%d\nOpen Windows\nLeave Room if
Necessary\n\n",gas);

}

}

```

```

Void setup(){
    Serial.begin(115200);

```

```
SerialBT.begin("ESP32_AirMonitor");
```

```
Dht.begin();
```

```
pinMode(RED_PIN,OUTPUT);
```

```
pinMode(GREEN_PIN,OUTPUT);
```

```
pinMode(BLUE_PIN,OUTPUT);
```

```
pinMode(BUZZER,OUTPUT);
```

```
Serial.println("Stabilizing MQ-2...");
```

```
Delay(10000);
```

```
Long sum=0;
```

```
For(int i=0;i<30;i++){
```

```
    Sum+=analogRead(MQ2_PIN);
```

```
    Delay(200);
```

```
}
```

```
Baseline=sum/30;
```

```
    Serial.println("Timestamp, MQ-2 Value, DHT11 Temp, DHT11 Humidity,  
Severity Level");
```

```
}
```

```
Void loop(){
```

```
    Int gasRaw=analogRead(MQ2_PIN);
```

```
Int diff=gasRaw-baseline;
```

```
Float temp=dht.readTemperature();
```

```
Float hum=dht.readHumidity();
```

```
If(isnan(temp)||isnan(hum)){
```

```
    Serial.println("DHT ERROR");
```

```
    Delay(2000);
```

```
    Return;
```

```
}
```

```
String status;
```

```
If(diff<80){
```

```
    Status="SAFE";
```

```
    setColor(LOW,HIGH,LOW);
```

```
    noTone(BUZZER);
```

```
}else if(diff<250){
```

```
    Status="MODERATE";
```

```
    setColor(HIGH,HIGH,LOW);
```

```
    tone(BUZZER,1000,200);
```

```
}else{
```

```
    Status="DANGER";
```

```
    setColor(HIGH,LOW,LOW);
```

```
tone(BUZZER,2000);  
}
```

```
Int sec=millis()/1000;
```

```
Char ts[6];
```

```
Sprintf(ts,"%02d:%02d",sec/60,sec%60);
```

```
Serial.print(ts);
```

```
Serial.print(", ");
```

```
Serial.print(gasRaw);
```

```
Serial.print(", ");
```

```
Serial.print(temp);
```

```
Serial.print(", ");
```

```
Serial.print(hum);
```

```
Serial.print(", ");
```

```
Serial.println(status);
```

```
sendBluetoothAlert(status,gasRaw,temp,hum);
```

```
delay(2000);
```

```
}
```

Q42. Automated Plant Watering System

Problem Statement

A J&K horticulture farmer wants to automate irrigation for a small saffron farm plot. The system should read soil moisture every 30 minutes and activate a water pump only when the soil is dry. Saffron is sensitive to overwatering — the system must also prevent watering if it rained recently (use LDR to detect low light/overcast sky as a rain proxy and also leverage the use of api also). The farmer must be able to manually override via a button.

Requirements:

- Soil moisture sensor reading with dry/wet/optimal zones
- Relay module controlling water pump (simulate with LED)
- LDR-based rain proxy logic
- Manual override button with 5-minute lockout after manual activation
- Display system status on 16x2 LCD: moisture%, mode, last watered time (millis-based)
- ESP8266 Wi-Fi logging to Google Sheets via IFTTT/ Google Scripts similar

Answer42This project uses an **ESP32** to monitor soil moisture, ambient light, and weather conditions to determine whether irrigation is required.

For simulation in Wokwi, a **potentiometer** is used in place of a soil moisture sensor, as a dedicated soil moisture sensor is not available in the simulator. An **LDR** serves as a local rain indicator by detecting low light or overcast conditions, while the **OpenWeatherMap REST API** provides current weather information to check if rainfall is occurring in the selected location.

The irrigation pump is represented by an **LED** connected to the ESP32. Watering takes place only when all of the following conditions are satisfied:

- Soil moisture is classified as **DRY**
- The LDR detects **no overcast conditions**
- The weather API indicates **no rain**

A push button allows the farmer to start irrigation manually whenever needed. Once manual watering is triggered, the system enters a **5-minute lockout period** to prevent repeated watering and reduce the risk of over-irrigation.

A **16×2 I2C LCD** continuously shows the soil moisture percentage, the current operating mode, and the time elapsed since the last watering cycle.

All sensor readings are uploaded to **Google Sheets** through Google Apps Script using a Wi-Fi connection.

✂ Hardware Components

Component	Quantity
----- :-----☺	
ESP32 Dev Module	1
Potentiometer *(Soil Moisture Simulation)*	1
LDR	1
Push Button	1

| Green LED *(Pump Simulation)* | 1 |

| 220Ω Resistor | 1 |

| 16×2 LCD (I2C) | 1 |

| Jumper Wires | As required |

Pin Connections

| Component | ESP32 Pin |

--	--

| Potentiometer | GPIO34 |

| LDR | GPIO35 |

| Push Button | GPIO18 |

| Pump LED | GPIO19 |

| LCD SDA | GPIO21 |

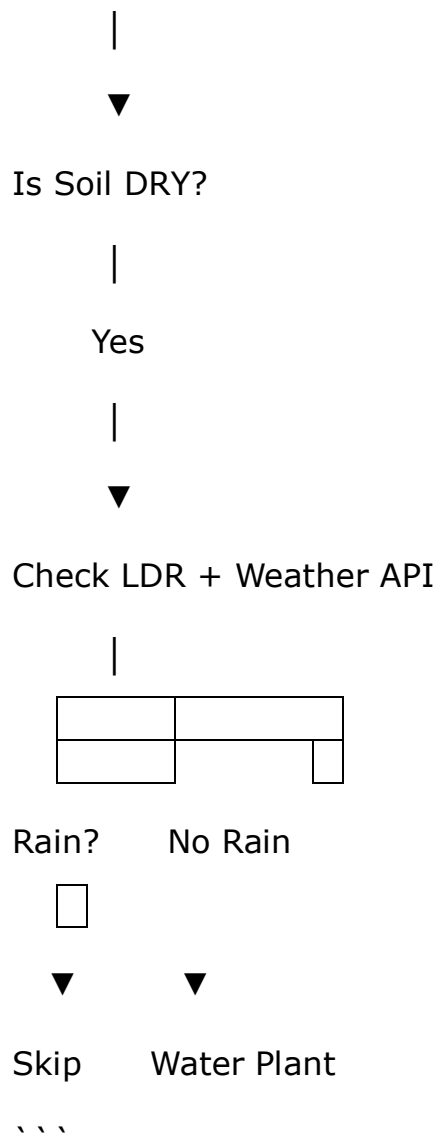
| LCD SCL | GPIO22 |

Irrigation Logic

The irrigation process follows the decision flow shown below:

```text

Read Soil Moisture



---

 Soil Moisture Zones

| Moisture (%)   Status |                                                                                             |
|-----------------------|---------------------------------------------------------------------------------------------|
|                       |                                                                                             |
| 0 – 29%               |  DRY     |
| 30 – 69%              |  OPTIMAL |

| 70 – 100% | 💧 WET |

---

## ☁ Rain Detection

The system relies on **two independent methods** to detect rainfall conditions:

- **LDR** senses low-light or overcast conditions.
- **OpenWeatherMap REST API** retrieves current weather information.

If **either method** detects rain, the irrigation process is skipped.

---

## 📺 LCD Display

Example:

```
` ``text
```

M:45% DRY

Last:120s

```
` ``
```

---

## ☁ Google Sheets Logging

Sensor data is uploaded using:

- Wi-Fi
- HTTP GET Requests
- Google Apps Script
- Google Sheets

Each record includes:

- Timestamp
  - Moisture Percentage
  - Moisture Status
  - Rain Status
  - Last Watered Time
- 

## 🚀 Features

- Automatic irrigation
- Soil moisture monitoring
- LDR-based rain detection

- Weather API integration
  - Manual watering override
  - 5-minute safety lockout
  - LCD status display
  - Google Sheets cloud logging
  - Offline decision making using LDR when internet is unavailable
- 

### ⚠ Simulation Notes

This project was developed using **Wokwi**.

To overcome simulator limitations:

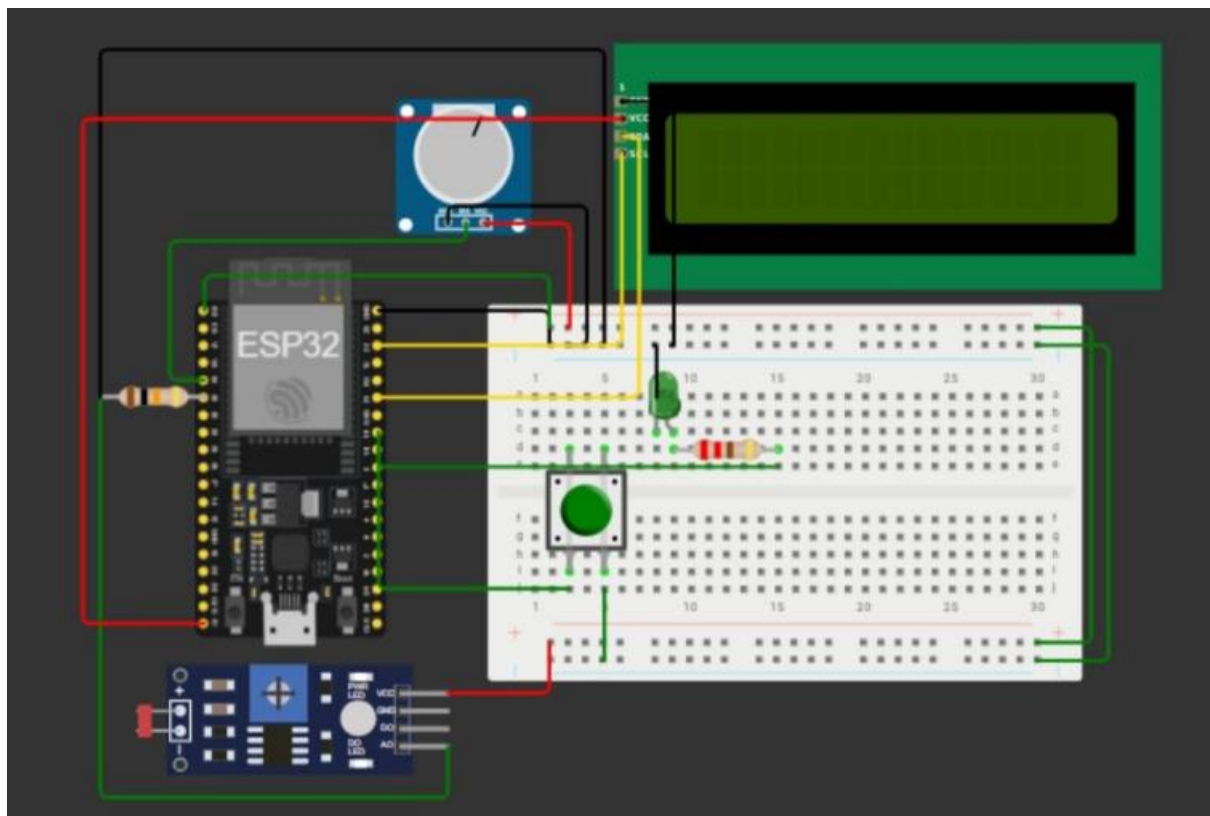
- A **potentiometer** is used to simulate the soil moisture sensor.
- A **green LED** represents the relay-controlled water pump.
- An **ESP32** is used instead of an ESP8266 because the project requires **two analog input pins** for the soil moisture simulation and the LDR, whereas the ESP8266 provides only one ADC pin.

These substitutions maintain the intended functionality of the system while allowing the entire project to be simulated successfully.

---

### 📖 Libraries Used

- WiFi.h
- HTTPClient.h
- ArduinoJson
- Wire.h
- LiquidCrystal\_I2C



## Code

```
#include <WiFi.h>
```

```
#include <HttpClient.h>
```

```
#include <ArduinoJson.h>
```

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
// WiFi Credentials
```

```
Const char* WIFI_SSID = "Wokwi-GUEST";
```

```
Const char* WIFI_PASSWORD = "";
```

```
// OpenWeather API
```

```
Const char* API_KEY = "YOUR API KEY";
```

```
Const char* CITY = "Jammu";
```

```
// Google Apps Script URL
```

```
Const char* SCRIPT_URL =
```

```
"YOUR SCRIPT URL";
```

```
// Pins
```



```
#define SOIL_PIN 34
```

```
#define LDR_PIN 35
```

```
#define BUTTON_PIN 18
```

```
#define PUMP_PIN 19
```

```
Unsigned long previousRead = 0;
```

```
Const unsigned long READ_INTERVAL = 30000;
```

```
Unsigned long lastWatered = 0;
```

```
Unsigned long manualOverrideTime = 0;
```

```
Const unsigned long LOCKOUT = 300000;
```

```
Bool rainAPI = false;
```

```
Bool rainLDR = false;
```

```
Bool pumpState = false;
```

```
String moistureState = "";
```

```
Int moistureRaw = 0;
```

```
Int moisturePercent = 0;
```

```
Int lightRaw = 0;
```

```
// WiFi
```

```
Void connectWiFi()
```

```
{
```

```
 Serial.print("Connecting");
```

```
 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
 While (WiFi.status() != WL_CONNECTED)
```

```
 {
```

```
 Serial.print(".");
```

```
 Delay(500);
```

```
 }
```

```
 Serial.println();
```

```
Serial.println("WiFi Connected");

}

// Weather API

Bool checkWeather()
{

 HTTPClient http;

 String url =
http://api.openweathermap.org/data/2.5/weather?q=Jammu&appid= +
 String(CITY) +
 "&appid=" +
 String(API_KEY);

 http.begin(url);

 int code = http.GET();

 if(code == 200)
 {
```

```
DynamicJsonDocument doc(2048);
```

```
deserializeJson(doc, http.getString());
```

```
String weather =
```

```
Doc["weather"][0]["main"];
```

```
http.end();
```

```
if(weather == "Rain")
```

```
 return true;
```

```
if(weather == "Drizzle")
```

```
 return true;
```

```
if(weather == "Thunderstorm")
```

```
 return true;
```

```
return false;
```

```
}
```

```
http.end();
```

```
 return false;

}

// Read Soil Moisture

Void readSoil()
{

 moistureRaw = analogRead(SOIL_PIN);

 moisturePercent = map(moistureRaw, 0, 4095, 0, 100);

 if(moisturePercent < 30)
 moistureState = "DRY";

 else if(moisturePercent < 70)
 moistureState = "OPTIMAL";

 else
 moistureState = "WET";

}
```

```
// Read LDR
```

```
Void readLDR()
```

```
{
```

```
 lightRaw = analogRead(LDR_PIN);
```

```
 if(lightRaw < 1500)
```

```
 rainLDR = true;
```

```
 else
```

```
 rainLDR = false;
```

```
}
```

```
// Water Plant
```

```
Void waterPlant()
```

```
{
```

```
 pumpState = true;
```

```
 digitalWrite(PUMP_PIN, HIGH);
```

```
lcd.clear();
```

```
lcd.setCursor(0,0);
```

```
lcd.print("Pump Running");
```

```
lcd.setCursor(0,1);
```

```
lcd.print("Watering...");
```

```
delay(5000);
```

```
digitalWrite(PUMP_PIN, LOW);
```

```
pumpState = false;
```

```
lastWatered = millis();
```

```
}
```

```
// LCD Display
```

```
Void updateLCD()
```

```
{
```

```
Lcd.clear();
```

```
Lcd.setCursor(0,0);
```

```
Lcd.print("M:");
```

```
Lcd.print(moisturePercent);
```

```
Lcd.print("% ");
```

```
Lcd.print(moistureState);
```

```
Lcd.setCursor(0,1);
```

```
Lcd.print("Last:");
```

```
Lcd.print(lastWatered/1000);
```

```
Lcd.print("s");
```

```
}
```

```
// Serial Output
```



```
Void printStatus()
{

 Serial.println("-----");

 Serial.print("Moisture : ");
 Serial.print(moisturePercent);
 Serial.println("%");

 Serial.print("State : ");
 Serial.println(moistureState);

 Serial.print("LDR Rain : ");

 If(rainLDR)
 Serial.println("YES");

 Else
 Serial.println("NO");

 Serial.print("Weather API Rain : ");

 If(rainAPI)
 Serial.println("YES");
```

Else

Serial.println("NO");

Serial.print("Last Watered : ");

Serial.print(lastWatered/1000);

Serial.println(" sec");

}

// Google Sheets Logging

Void logGoogleSheets()

{

If(WiFi.status() != WL\_CONNECTED)

Return;

HTTPClient http;

String url =

```
String(SCRIPT_URL) +
```

```
"?moisture=" +
```

```
String(moisturePercent) +
```

```
"&status=" +
```

```
moistureState +
```

```
"&rain=" +
```

```
String(rainAPI);
```

```
http.begin(url);
```

```
http.GET();
```

```
http.end();
```

```
}
```

```
// Manual Override
```

```
Void manualOverride()
{

 If(digitalRead(BUTTON_PIN) == LOW)
 {

 If(millis() - manualOverrideTime >= LOCKOUT)
 {

 manualOverrideTime = millis();

 Serial.println("Manual Override Activated");

 waterPlant();

 }

 Else
 {

 Serial.println("Manual Override Locked");

 }

 }

}
```

```
}
```

```
}
```

```
// SETUP
```

```
Void setup()
```

```
{
```

```
 Serial.begin(115200);
```

```
 Lcd.init();
```

```
 Lcd.backlight();
```

```
 pinMode(PUMP_PIN, OUTPUT);
```

```
 pinMode(BUTTON_PIN, INPUT_PULLUP);
```

```
 digitalWrite(PUMP_PIN, LOW);
```

```
 connectWiFi();
```

```
 lcd.clear();
```

```
lcd.setCursor(0,0);
```

```
lcd.print("Plant Watering");
```

```
lcd.setCursor(0,1);
```

```
lcd.print("System Ready");
```

```
delay(2000);
```

```
}
```

```
// LOOP
```

```
Void loop()
```

```
{
```

```
// Manual Override
```

```
manualOverride();
```

```
// Read every 30 seconds
```

```
// (30 minutes on real hardware)
```

```
If(millis() - previousRead >= READ_INTERVAL)
```

```
{
```

```
 previousRead = millis();
```

```
 readSoil();
```

```
 readLDR();
```

```
 rainAPI = checkWeather();
```

```
 updateLCD();
```

```
 printStatus();
```

```
 logGoogleSheets();
```

```
 if(moistureState == "DRY")
```

```
 {
```

```
 If(!rainLDR && !rainAPI)
```

```
 {
```

```
Serial.println("Conditions OK");
```

```
Serial.println("Starting Pump");
```

```
waterPlant();
```

```
}
```

```
Else
```

```
{
```

```
Serial.println("Watering Skipped");
```

```
Serial.println("Rain Detected");
```

```
}
```

```
}
```

```
Else if(moistureState == "OPTIMAL")
```

```
{
```

```
Serial.println("Soil Moisture Optimal");
```



```
}
```

```
Else
```

```
{
```

```
 Serial.println("Soil Already Wet");
```

```
}
```

```
Serial.println();
```

```
}
```

```
}
```

```
.
```

#### **Q43. Smart Door Lock with OTP**

##### **Problem Statement**

A university hostel wants to replace physical keys with a PIN-based entry system that also generates a One-Time Password. Design a system where a student enters their 4-digit ID on a keypad, the system generates a 6-digit OTP, sends it over Bluetooth to their phone, and only grants access if they enter the OTP within 30 seconds.

#### Requirements:

- 4x4 keypad for ID and OTP entry
- 16x2 LCD or 0.96 OLED for prompts and masked input (show for digits)
- HC-05 Bluetooth: Send OTP string to connected phone / Alternate use ESP32
- Servo motor: rotate 90° to simulate door unlock
- 30-second countdown timer using millis() (non-blocking) // Don't use delay
- Log all access attempts (success/failure) to Serial Monitor

**Answer 43** The system is built around an **ESP32** connected to a **4×4 keypad**, **16×2 I2C LCD**, and an **SG90 servo motor**.

The user begins by entering a **4-digit ID** using the keypad. Once a valid ID is received, the ESP32 creates a **random 6-digit OTP** and notifies the user that the OTP has been generated. In an actual hardware setup, this OTP can be sent to a smartphone using the ESP32's built-in Bluetooth through the **BluetoothSerial** library or an **HC-05 Bluetooth module**. Since Wokwi does not support Bluetooth communication, the OTP is displayed on the Serial Monitor for demonstration.

The generated OTP must be entered within **30 seconds**. A non-blocking timer implemented with `millis()` keeps track of the remaining time. If the correct OTP is entered before the timer expires, the servo motor rotates **90°** to simulate unlocking the door and then returns to its original position to lock the door again. If the OTP is incorrect or the time limit expires, access is denied and the system automatically resets for the next user.

Every login attempt, whether successful or unsuccessful, is recorded on the Serial Monitor for monitoring and debugging.

---

🔧 Hardware Components

| Component         | Quantity    |
|-------------------|-------------|
| ----- :-----☹     |             |
| ESP32 Dev Module  | 1           |
| 4×4 Matrix Keypad | 1           |
| 16×2 I2C LCD      | 1           |
| SG90 Servo Motor  | 1           |
| Jumper Wires      | As required |

---

🔌 Pin Connections

| Component | ESP32 Pin |
|-----------|-----------|
|           |           |
| Keypad R1 | GPIO13    |
| Keypad R2 | GPIO12    |
| Keypad R3 | GPIO14    |
| Keypad R4 | GPIO27    |
| Keypad C1 | GPIO26    |
| Keypad C2 | GPIO25    |
| Keypad C3 | GPIO33    |
| Keypad C4 | GPIO32    |
| LCD SDA   | GPIO21    |

| LCD SCL | GPIO22 |

| Servo Signal | GPIO18 |

---

 System Workflow

```text

Power On

|



Enter 4-digit ID

|



Generate 6-digit OTP

|



OTP Sent

(Bluetooth / Serial Monitor)

|



Enter OTP

|

| | |
|--|--|
| | |
| | |

Correct Incorrect / Timeout



Unlock Access Denied

Door Reset System

...

Features

- 4-digit student ID authentication
 - Random 6-digit OTP generation
 - 30-second OTP expiry timer using ``millis()``
 - Servo motor door lock simulation
 - LCD prompts and masked keypad input
 - Serial logging of access attempts
 - Bluetooth-ready implementation for physical ESP32
 - Automatic system reset after each attempt
-

LCD Screens

Startup

```text

Smart Door Lock

System Start

```

ID Entry

```text

Enter ID

.....

```

OTP Entry

```text

Enter OTP

.....

```

Success

```text

ACCESS

GRANTED

```

Failure

```text

ACCESS

DENIED

```

Timeout

```text

OTP

EXPIRED

```

 Sample Serial Output

```text

System Ready

ID Entered: 1234

OTP Generated: 548271

OTP Entered: 548271

ACCESS GRANTED

Door Unlocked

---

LOG: SUCCESS ENTRY

` ``

Failed attempt:

` ``text

System Ready

ID Entered: 1234

OTP Entered: 123456

ACCESS DENIED

---

LOG: FAILED ENTRY



\\ \

---

## Bluetooth Support

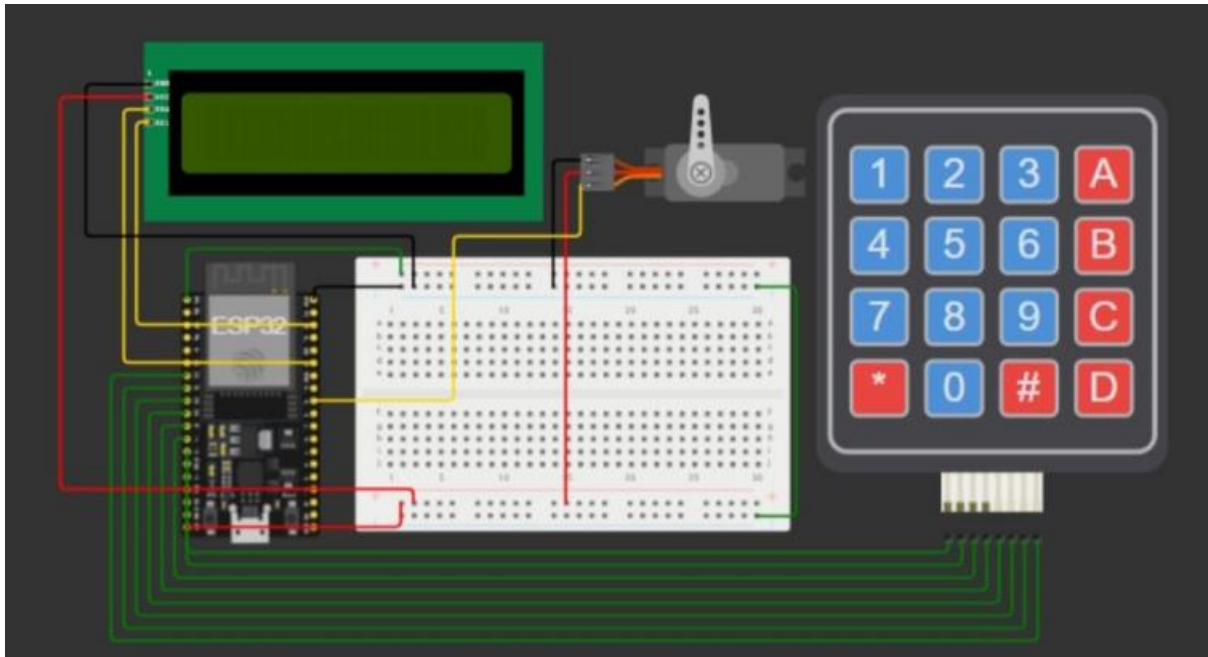
In a physical ESP32 implementation, the generated OTP can be sent to a connected smartphone using the **BluetoothSerial** library or an **HC-05** Bluetooth module.

**Simulation Note:** Since Wokwi does not currently support Bluetooth communication, the OTP is displayed on the Serial Monitor. A sample mobile notification is also included to demonstrate how the system would behave on real hardware.

---

## Libraries Used

- Wire.h
- LiquidCrystal\_I2C.h
- Keypad.h
- ESP32Servo.h



## **Code**

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <Keypad.h>

#include <ESP32Servo.h>

/* ----- LCD Display ----- */

// If your LCD doesn't respond, try changing 0x27 to 0x3F
LiquidCrystal_I2C lcd(0x27, 16, 2);

/* ----- Servo Motor ----- */

Servo doorServo;

#define SERVO_PIN 18
```

```
/* ----- Keypad ----- */
```

```
Const byte ROWS = 4;
```

```
Const byte COLS = 4;
```

```
Char keys[ROWS][COLS] = {
```

```
 {'1','2','3','A'},
```

```
 {'4','5','6','B'},
```

```
 {'7','8','9','C'},
```

```
 {'*','0','#','D'}
```

```
};
```

```
Byte rowPins[ROWS] = {13, 12, 14, 27};
```

```
Byte colPins[COLS] = {26, 25, 33, 32};
```

```
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS,
COLS);
```

```
/* ----- Global Variables ----- */
```

```
String userID = "";
```

```
String otp = "";
```

```
String inputOTP = "";
```

```
Bool otpSent = false;
```

```
Unsigned long otpStartTime = 0;

Const unsigned long otpTimeout = 30000;

/* ----- Setup ----- */

Void setup() {

 Serial.begin(115200);

 // Initialize servo (door locked by default)
 doorServo.attach(SERVO_PIN);
 doorServo.write(0);

 // Initialize LCD
 Lcd.init();
 Lcd.backlight();

 Lcd.setCursor(0,0);
 Lcd.print("Smart Door Lock");
 Lcd.setCursor(0,1);
 Lcd.print("System Start");

 Delay(2000);

 Lcd.clear();
```

```

Serial.println("System Ready");
}

/* ----- Generate Random OTP ----- */
String generateOTP() {

 String code = "";

 For (int I = 0; I < 6; i++) {
 Code += String(random(0, 9));
 }

 Return code;
}

/* ----- Main Program ----- */
Void loop() {

 Char key = keypad.getKey();

 /* ----- Step 1 : User ID Entry ----- */
 If (!otpSent) {

```

```
Lcd.setCursor(0,0);
```

```
Lcd.print("Enter ID (4 digit)");
```

```
If (key) {
```

```
 If (key >= '0' && key <= '9') {
```

```
 userID += key;
```

```
 lcd.setCursor(userID.length() - 1, 1);
```

```
 lcd.print("*");
```

```
 Serial.print("*");
```

```
 }
```

```
If (userID.length() == 4) {
```

```
 Serial.println("\nID Entered: " + userID);
```

```
 Otp = generateOTP();
```

```
 Serial.println("OTP GENERATED: " + otp);
```

```
Lcd.clear();
```

```
Lcd.setCursor(0,0);
Lcd.print("OTP Sent!");
Lcd.setCursor(0,1);
Lcd.print("Check Serial");
```

```
otpSent = true;
otpStartTime = millis();
```

```
Serial.println("Enter OTP:");
```

```
}

}

}
```

```
/* ----- Step 2 : OTP Verification ----- */
```

```
Else {
```

```
 If (millis() – otpStartTime > otpTimeout) {
```

```
 Serial.println("OTP EXPIRED!");
```

```
 Lcd.clear();
```

```
 Lcd.print("OTP Expired");
```

```
 resetSystem(false);
```

```
return;
```

```
}
```

```
Lcd.setCursor(0,0);
```

```
Lcd.print("Enter OTP:");
```

```
If (key) {
```

```
 If (key >= '0' && key <= '9') {
```

```
 inputOTP += key;
```

```
 lcd.setCursor(inputOTP.length() - 1, 1);
```

```
 lcd.print("*");
```

```
 Serial.print("*");
```

```
 }
```

```
If (inputOTP.length() == 6) {
```

```
 Serial.println("\nOTP ENTERED: " + inputOTP);
```

```
 If (inputOTP == otp) {
```



```
Serial.println("ACCESS GRANTED");
```

```
Lcd.clear();
```

```
Lcd.print("ACCESS GRANTED");
```

```
doorServo.write(90);
```

```
delay(3000);
```

```
doorServo.write(0);
```

```
Serial.println("Door Unlocked");
```

```
resetSystem(true);
```

```
}
```

```
Else {
```

```
Serial.println("ACCESS DENIED");
```

```
Lcd.clear();
```

```
Lcd.print("ACCESS DENIED");
```

```
resetSystem(false);
```

```
}
```

```
}
```

```
}
```

```

 }
}

/* ----- Reset System ----- */
Void resetSystem(bool success) {

 Serial.println("-----");

 If (success)
 Serial.println("LOG: SUCCESS ENTRY");
 Else
 Serial.println("LOG: FAILED ENTRY");

 userID = "";
 inputOTP = "";
 otp = "";
 otpSent = false;

 delay(1500);
 lcd.clear();
}

```

#### **Q44.** PIR-Based Security Camera Trigger

Problem Statement

A small shop in Haryana, Rohtak wants a low-cost security alert system. When motion is detected after shop hours (manually configured time window), the system should sound an alarm, flash lights, and optionally send a notification. Build the motion-detection and alerting module.

Requirements:

- PIR sensor with adjustable sensitivity (via potentiometer mapped to threshold)
- Active buzzer alarm with 3-stage escalation (warning → alarm → urgent)
- LED strip simulation (3 LEDs flashing in sequence)
- Time-window configuration via Serial command (e.g., SET\_HOURS 22 06)
- Log motion events with timestamps to Serial Monitor
- ESP32 sends Telegram bot message on motion detection

**Answer44:** An **ESP32** continuously monitors a PIR motion sensor to detect movement. Motion events are processed only during the user-defined monitoring hours, which can be configured through the Serial Monitor using the `SET_HOURS`` command.

A potentiometer is included to simulate adjustable sensor sensitivity. Based on the potentiometer reading, the system classifies the detected motion into one of three alert levels:

- **Warning**
- **Alarm**
- **Urgent**

Each alert level activates a unique combination of LED indications and buzzer tones to represent the severity of the detected motion.

Whenever movement is detected within the configured monitoring period, the ESP32 logs the event along with its timestamp to the Serial Monitor.

For deployment on a physical ESP32, Telegram notification support is provided through a separate integration file, allowing motion alerts to be sent directly to the owner's Telegram account.

---

## 🔧 Hardware Components

| Component         | Quantity    |
|-------------------|-------------|
| ----- :-----☺     |             |
| ESP32 Dev Module  | 1           |
| PIR Motion Sensor | 1           |
| Potentiometer     | 1           |
| Active Buzzer     | 1           |
| Green LED         | 1           |
| Yellow LED        | 1           |
| Red LED           | 1           |
| 220Ω Resistors    | 3           |
| Jumper Wires      | As required |

---

## 🔌 Pin Connections

| Component | ESP32 Pin |

|  |  |
|--|--|
|  |  |
|--|--|

| PIR OUT | GPIO27 |

| Potentiometer | GPIO34 |

| Active Buzzer | GPIO25 |

| Green LED | GPIO18 |

| Yellow LED | GPIO19 |

| Red LED | GPIO21 |

---

## ⚙ Features

- PIR motion detection
- Adjustable sensitivity using potentiometer
- Three-stage alarm escalation
- Sequential LED alert simulation
- Active buzzer warning
- Configurable monitoring hours
- Timestamped Serial logging
- Telegram integration for physical ESP32 deployment

---

## 🔔 Alarm Levels

| Level | LEDs | Buzzer |

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

| Warning | Green | Low frequency |

| Alarm | Green + Yellow | Medium frequency |

| Urgent | Green + Yellow + Red | High frequency with rapid flashing |

---

## Monitoring Hours

Monitoring hours can be configured through the Serial Monitor.

Example:

```
```text
```

```
SET_HOURS 22 06
```

```
```
```

This enables motion monitoring from **\*\*10:00 PM until 6:00 AM\*\***.

---

## Sample Serial Output

```
```text
```

PIR Security Camera Trigger

Monitoring Hours : 22 -> 6

Timestamp : 18 sec

Motion Detected

Sensitivity Value : 3450

Alert Level : URGENT

[18 sec] URGENT

Motion Event Logged

...

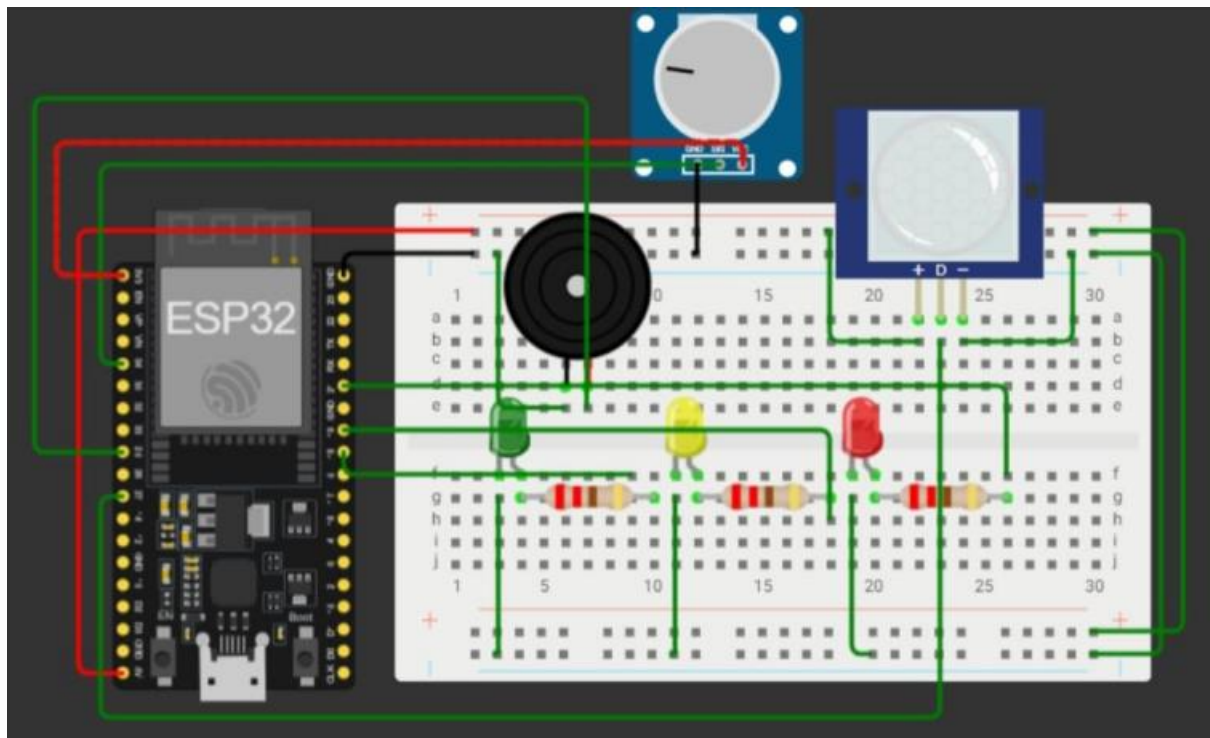
Telegram Integration

A separate file named `**`TelegramIntegration.ino`**` is included in the repository. It contains the Wi-Fi connection and Telegram Bot API implementation needed to send motion alerts to a Telegram chat when using a physical ESP32.

****Simulation Note:**** The Wokwi simulation demonstrates PIR-based motion detection, alarm escalation, LED indications, and Serial logging. Telegram messaging is provided separately for use on actual hardware.

Libraries Used

- WiFi.h
- HTTPClient.h (Telegram integration)



Code pir security


```
#include <WiFi.h>
```

```
/* ----- Pin Configuration ----- */
```

```
#define PIR_PIN 27
```

```
#define BUZZER_PIN 25
```

```
#define LED_GREEN 18
```

```
#define LED_YELLOW 19
```

```
#define LED_RED 21
```

```
// Potentiometer used to adjust alert sensitivity
```

```
#define POT_PIN 34
```

```
/* ----- Monitoring Time ----- */
```

```
Int starthour = 0;
```

```
Int endhour = 24;
```

```
Bool windowSet = false;
```

```
// Simulated current hour
```

```
Int currentHour = 12;
```

```
/* ----- Global Variables ----- */
```

```
Unsigned long lastTrigger = 0;
```

```
Int alertLevel = 1;
```

```
/* ----- Buzzer Function ----- */
```

```
Void beep(int freq, int duration)
```

```
{
```

```
    ledcWriteTone(BUZZER_PIN, freq);
```

```
    delay(duration);
```

```
    ledcWriteTone(BUZZER_PIN, 0);
```

```
}
```

```
/* ----- Read Serial Command ----- */
```

```
Void readCommand()
```

```
{
```

```
    If (Serial.available())
```

```
    {
```

```
        String cmd = Serial.readStringUntil('\n');
```

```
        If (cmd.startsWith("SET_HOURS"))
```

```
        {
```

```
            Sscanf(cmd.c_str(),
```

```
                "SET_HOURS %d %d",
```

```
                &startHour,
```

```
                &endHour);
```

```

    windowSet = true;

    Serial.print("Monitoring Hours: ");
    Serial.print(startHour);
    Serial.print(" -> ");
    Serial.println(endHour);
}
}
}

/* ----- Check Active Time ----- */
Bool isActiveTime()
{
    If (!windowSet)
        Return true;

    If (startHour < endHour)
    {
        Return (currentHour >= startHour &&
                currentHour < endHour);
    }

    Return (currentHour >= startHour ||

```

```

        currentHour < endHour);
    }

/* ----- Alert Handler ----- */
Void alertSystem(int level)
{
    Serial.print("[");
    Serial.print(millis() / 1000);
    Serial.print(" sec] ");

    If (level == 1)
    {
        Serial.println("WARNING");

        digitalWrite(LED_GREEN, HIGH);

        beep(1000, 200);

        digitalWrite(LED_GREEN, LOW);
    }
    Else if (level == 2)
    {
        Serial.println("ALARM");
    }
}

```

```
digitalWrite(LED_GREEN, HIGH);  
digitalWrite(LED_YELLOW, HIGH);
```

```
beep(1600, 350);
```

```
digitalWrite(LED_GREEN, LOW);  
digitalWrite(LED_YELLOW, LOW);
```

```
}
```

```
Else
```

```
{
```

```
Serial.println("URGENT");
```

```
For (int I = 0; I < 4; i++)
```

```
{
```

```
digitalWrite(LED_GREEN, HIGH);  
digitalWrite(LED_YELLOW, HIGH);  
digitalWrite(LED_RED, HIGH);
```

```
beep(2500, 150);
```

```
digitalWrite(LED_GREEN, LOW);  
digitalWrite(LED_YELLOW, LOW);  
digitalWrite(LED_RED, LOW);
```

```

        delay(100);
    }
}
}

/* ----- Setup ----- */

Void setup()
{
    Serial.begin(115200);

    pinMode(PIR_PIN, INPUT);
    pinMode(POT_PIN, INPUT);

    pinMode(LED_GREEN, OUTPUT);
    pinMode(LED_YELLOW, OUTPUT);
    pinMode(LED_RED, OUTPUT);

    // Configure buzzer using PWM
    ledcAttach(BUZZER_PIN, 2000, 8);

    Serial.println("=====");
    Serial.println(" PIR Security Camera Trigger");
    Serial.println("=====");
    Serial.println("Command Example:");

```

```
Serial.println("SET_HOURS 22 06");

Serial.println();

}

/* ----- Main Loop ----- */

Void loop()

{

    // Check for commands from Serial Monitor

    readCommand();


    // Read potentiometer value

    Int potValue = analogRead(POT_PIN);


    If (potValue < 1365)

        alertLevel = 1;


    else if (potValue < 2730)

        alertLevel = 2;


    else

        alertLevel = 3;


    // Read PIR sensor status

    Int pirState = digitalRead(PIR_PIN);
```

```
If (pirState == HIGH && isActiveTime())
{
    If (millis() - lastTrigger > 3000)
    {
        lastTrigger = millis();

        Serial.println("-----");

        Serial.print("Timestamp : ");
        Serial.print(millis() / 1000);
        Serial.println(" sec");

        Serial.println("Motion Detected");

        Serial.print("Sensitivity Value : ");
        Serial.println(potValue);

        Serial.print("Alert Level : ");

        Switch (alertLevel)
        {
            Case 1:
                Serial.println("WARNING");
```



```
Break;
```

```
Case 2:
```

```
Serial.println("ALARM");
```

```
Break;
```

```
Case 3:
```

```
Serial.println("URGENT");
```

```
Break;
```

```
}
```

```
alertSystem(alertLevel);
```

```
Serial.println("Motion Event Logged");
```

```
Serial.println("-----");
```

```
}
```

```
}
```

```
// Small delay to reduce CPU usage
```

```
Delay(100);
```

```
}
```

Code telegram integration

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
/* ----- Wi-Fi Credentials ----- */
```

```
Const char* WIFI_SSID = "YOUR_WIFI";
```

```
Const char* WIFI_PASSWORD = "YOUR_PASSWORD";
```

```
/* ----- Telegram Bot Details ----- */
```

```
Const char* BOT_TOKEN = "YOUR_BOT_TOKEN";
```

```
Const char* CHAT_ID  = "YOUR_CHAT_ID";
```

```
/* ----- Connect to Wi-Fi ----- */
```

```
Void connectWiFi()
```

```
{
```

```
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
    Serial.print("Connecting to Wi-Fi");
```

```
    While (WiFi.status() != WL_CONNECTED)
```

```

{
    Delay(500);

    Serial.print(".");
}

Serial.println();

Serial.println("Wi-Fi Connected");
}

/* ----- Send Telegram Alert ----- */
Void sendTelegramAlert(String level)
{
    If (WiFi.status() != WL_CONNECTED)
        Return;

    HTTPClient http;

    String msg =
        "\ud83d\udc05 Motion Detected%0A"
        "Level: " + level +
        "%0ATime: " + String(millis() / 1000) + " sec";

    String url =
        https://api.telegram.org/bot +

```

```
String(BOT_TOKEN) +  
"/sendMessage?chat_id=" +  
String(CHAT_ID) +  
&text=" +  
Msg;  
  
http.begin(url);  
  
int httpCode = http.GET();  
  
if (httpCode > 0)  
{  
    Serial.println("Telegram alert sent.");  
}  
Else  
{  
    Serial.println("Failed to send Telegram alert.");  
}  
  
http.end();  
}
```

Q45. Smart Greenhouse Controller

Problem Statement

Floriculture farmers in Kashmir growing tulips and roses need automated greenhouse climate control. Design a multi-parameter monitoring and control system that maintains optimal growing conditions: Temperature 18–22°C, Humidity 60–70%, and sufficient light for 12 hours per day.

Requirements:

- DHT11: Monitor temperature and humidity
- LDR: Monitor light levels with running average over 10 samples
- 3 relay outputs: Heater, Ventilation Fan, Grow Light (simulate with LEDs)
- PID-like control logic (simple bang-bang with hysteresis)
- 16x2 LCD: Cycle through all readings every 3 seconds
- BONUS: MQTT publish of all parameters to cloud dashboard

Answer45:

. An **ESP32** serves as the central controller of the greenhouse. A **DHT11 sensor** measures temperature and humidity, while an **LDR** continuously monitors ambient light intensity. To reduce noise from the light sensor, a **running average of the last 10 LDR readings** is calculated before making any control decisions.

The greenhouse uses a simple **bang-bang control algorithm with hysteresis** to maintain stable environmental conditions.

- If the temperature falls below the desired range, the heater is activated.

- If the temperature rises above the desired range, the ventilation fan is activated.
- If light intensity is insufficient, the grow light is switched on.

Three LEDs simulate the relay-controlled heater, fan, and grow light.

A **16×2 I2C LCD** cycles through sensor readings and actuator status every three seconds. All environmental parameters are also published to a public MQTT broker, allowing remote monitoring through any MQTT client.

✂ Hardware Components

Component	Quantity
----- :-----☺	
ESP32 Dev Module	1
DHT11 Sensor	1
LDR	1
10kΩ Resistor	1
16×2 I2C LCD	1
Green LED (Heater)	1
Yellow LED (Ventilation Fan)	1
White LED (Grow Light)	1
220Ω Resistors	3
Jumper Wires	As required

Pin Connections

Component	ESP32 Pin
-----------	-----------

--	--

DHT11 DATA	GPIO4
------------	-------

LDR	GPIO34
-----	--------

Heater LED	GPIO25
------------	--------

Fan LED	GPIO26
---------	--------

Grow Light LED	GPIO27
----------------	--------

LCD SDA	GPIO21
---------	--------

LCD SCL	GPIO22
---------	--------

Target Greenhouse Conditions

Parameter	Target Range
-----------	--------------

--	--

Temperature	**18°C – 22°C**
-------------	-----------------

Humidity	**60% – 70%**
----------	---------------

Light	Adequate for approximately **12 hours/day**
-------	---

⚙ Control Logic

The greenhouse controller uses ****bang-bang control with hysteresis****.

Temperature

- Temperature $< 17^{\circ}\text{C}$ → Heater ON
- Temperature $> 23^{\circ}\text{C}$ → Fan ON
- Temperature within range → Maintain current state

Humidity

- Humidity $> 75\%$ → Fan ON

Light

- Low light (LDR $<$ Threshold) → Grow Light ON
- Bright conditions → Grow Light OFF

LCD Display

The LCD automatically cycles every ****3 seconds****.

Screen 1

```text

Temp:21.5

Hum:65%

```

Screen 2

```text

Light:

1430

```

Screen 3

```text

H:1 F:0

L:1

```

The ESP32 publishes greenhouse parameters to the ****HiveMQ public broker****.

Published Topics

| Topic | Data |

--	--

| greenhouse/temp | Temperature |

| greenhouse/humidity | Humidity |

| greenhouse/light | Average LDR Value |

| greenhouse/heater | Heater Status |

| greenhouse/fan | Fan Status |

| greenhouse/lightStatus | Grow Light Status |

These topics can be monitored using MQTT Explorer or any MQTT dashboard.

🚀 Features

- Temperature monitoring
- Humidity monitoring
- Running average over 10 LDR samples
- Automatic heater control
- Automatic ventilation control

- Automatic grow light control
 - Bang-bang control with hysteresis
 - LCD status display
 - MQTT cloud publishing
 - Serial Monitor logging
-

Sample Serial Output

```text

----- GREENHOUSE DATA -----

Temperature: 20.8 °C

Humidity: 66 %

Light Avg: 1425

Heater: 0

Fan: 0

Light: 1

---

```

Libraries Used

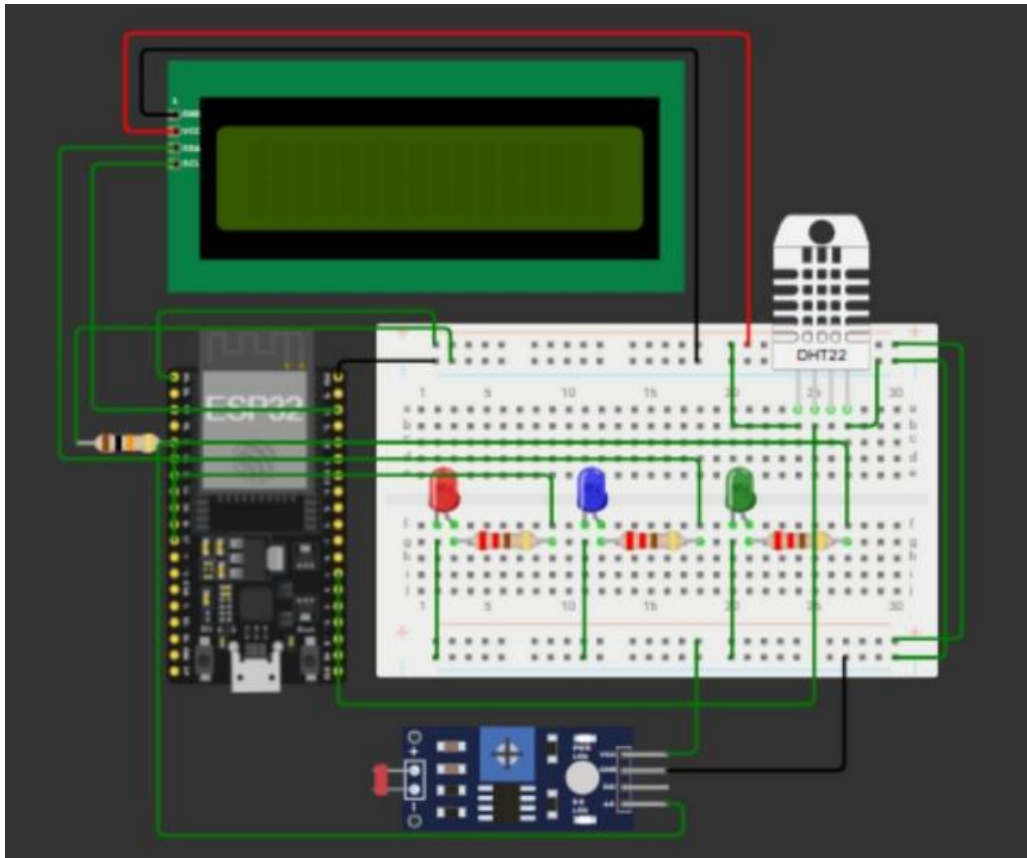
- WiFi.h
 - PubSubClient.h
 - Wire.h
 - LiquidCrystal_I2C.h
 - DHT.h
-

Simulation Notes

This project was developed using ****Wokwi****.

To simplify simulation:

- Relay modules are represented using LEDs.
- The LDR provides simulated ambient light values.- MQTT messages are published to the ****HiveMQ public broker**** for demonstration purposes.



Code

```
#include <WiFi.h>

#include <PubSubClient.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <DHT.h>

// ===== Wi-Fi Configuration =====

Const char* ssid = "Wokwi-GUEST";
```

```
Const char* password = "";

// ===== MQTT Configuration =====

Const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;

PubSubClient client(espClient);


// ===== Pin Definitions =====

#define DHTPIN 4

#define DHTTYPE DHT22

#define LDR_PIN 34


#define HEATER_PIN 25

#define FAN_PIN 26

#define LIGHT_PIN 27


// ===== Object Initialization =====

DHT dht(DHTPIN, DHTTYPE);

LiquidCrystal_I2C lcd(0x27, 16, 2);


// ===== Target Environmental Conditions =====

Float tempMin = 18, tempMax = 22;

Float humMin = 60, humMax = 70;
```

```
Float tempHyst = 1;
```

```
Float humHyst = 5;
```

```
// ===== LDR Sampling Buffer =====
```

```
Int samples[10];
```

```
Int idx = 0;
```

```
// ===== LCD Display Settings =====
```

```
Int mode = 0;
```

```
Unsigned long lastLCD = 0;
```

```
// ===== Timing Variables =====
```

```
Unsigned long lastPublish = 0;
```

```
// ===== Wi-Fi Connection Function =====
```

```
Void setup_wifi() {
```

```
    WiFi.begin(ssid, password);
```

```
    Serial.print("Connecting WiFi");
```

```
    While (WiFi.status() != WL_CONNECTED) {
```

```
        Delay(300);
```

```
        Serial.print(".");
```

```
    }
```

```
    Serial.println("\nWiFi Connected");
```

```
}
```

```
// ===== MQTT Reconnection Function =====
```

```
Void reconnect() {  
    While (!client.connected()) {  
        Serial.print("Connecting MQTT...");  
        If (client.connect("ESP32Greenhouse")) {  
            Serial.println("Connected");  
        } else {  
            Serial.print("Failed rc=");  
            Serial.print(client.state());  
            Delay(1000);  
        }  
    }  
}
```

```
// ===== LDR Average Reading Function =====
```

```
Int getLDR() {  
    Samples[idx] = analogRead(LDR_PIN);  
    Idx = (idx + 1) % 10;  
  
    Long sum = 0;  
    For (int I = 0; I < 10; i++) sum += samples[i];  
  
    Return sum / 10;
```



```
}
```

```
// ===== Greenhouse Control Logic =====
```

```
Void control(float t, float h, int l) {
```

```
    If (t < tempMin - tempHyst) {
```

```
        digitalWrite(HEATER_PIN, HIGH);
```

```
        digitalWrite(FAN_PIN, LOW);
```

```
    }
```

```
    Else if (t > tempMax + tempHyst) {
```

```
        digitalWrite(FAN_PIN, HIGH);
```

```
        digitalWrite(HEATER_PIN, LOW);
```

```
    }
```

```
    If (h > humMax + humHyst) {
```

```
        digitalWrite(FAN_PIN, HIGH);
```

```
    }
```

```
    If (l < 1500)
```

```
        digitalWrite(LIGHT_PIN, HIGH);
```

```
    else
```

```
        digitalWrite(LIGHT_PIN, LOW);
```

```
}
```

```
// ===== Data Publishing (MQTT & Serial Monitor) =====

Void sendData(float t, float h, int l) {

    // ----- Serial Monitor Output -----

    Serial.println("----- GREENHOUSE DATA -----");

    Serial.print("Temperature: "); Serial.print(t); Serial.println(" °C");

    Serial.print("Humidity: "); Serial.print(h); Serial.println(" %");

    Serial.print("Light Avg: "); Serial.println(l);


    Serial.print("Heater: "); Serial.println(digitalRead(HEATER_PIN));

    Serial.print("Fan: "); Serial.println(digitalRead(FAN_PIN));

    Serial.print("Light: "); Serial.println(digitalRead(LIGHT_PIN));

    Serial.println("-----\n");


    // ----- MQTT Data Publishing -----

    Client.publish("greenhouse/temp", String(t).c_str());

    Client.publish("greenhouse/humidity", String(h).c_str());

    Client.publish("greenhouse/light", String(l).c_str());

    Client.publish("greenhouse/heater",
String(digitalRead(HEATER_PIN)).c_str());

    Client.publish("greenhouse/fan", String(digitalRead(FAN_PIN)).c_str());

    Client.publish("greenhouse/lightStatus",
String(digitalRead(LIGHT_PIN)).c_str());

}
```

```
// ===== LCD Display Function =====
```

```
Void display(float t, float h, int l) {
```

```
    If (millis() - lastLCD > 3000) {
```

```
        Mode = (mode + 1) % 3;
```

```
        lastLCD = millis();
```

```
        lcd.clear();
```

```
    }
```

```
    If (mode == 0) {
```

```
        Lcd.setCursor(0,0);
```

```
        Lcd.print("Temp:");
```

```
        Lcd.print(t);
```

```
        Lcd.setCursor(0,1);
```

```
        Lcd.print("Hum:");
```

```
        Lcd.print(h);
```

```
    }
```

```
    Else if (mode == 1) {
```

```
        Lcd.setCursor(0,0);
```

```
        Lcd.print("Light:");
```

```
        Lcd.setCursor(0,1);
```

```
        Lcd.print(l);
```

```
    }
```

```
    Else {
```

```
Lcd.setCursor(0,0);  
Lcd.print("H:");  
Lcd.print(digitalRead(HEATER_PIN));  
Lcd.print(" F:");  
Lcd.print(digitalRead(FAN_PIN));  
Lcd.setCursor(0,1);  
Lcd.print("L:");  
Lcd.print(digitalRead(LIGHT_PIN));  
}  
}
```

```
// ===== System Setup =====
```

```
Void setup() {
```

```
    Serial.begin(115200);
```

```
    pinMode(HEATER_PIN, OUTPUT);
```

```
    pinMode(FAN_PIN, OUTPUT);
```

```
    pinMode(LIGHT_PIN, OUTPUT);
```

```
    dht.begin();
```

```
    lcd.init();
```

```
    lcd.backlight();
```

```
setup_wifi();

client.setServer(mqtt_server, 1883);


for (int I = 0; I < 10; i++) samples[i] = 0;
}


// ===== Main Program Loop =====

Void loop() {

    If (!client.connected()) reconnect();
    Client.loop();

    Float temp = dht.readTemperature();
    Float hum = dht.readHumidity();

    If (isnan(temp) || isnan(hum)) return;

    Int light = getLDR();

    Control(temp, hum, light);
    Display(temp, hum, light);

    // ----- Publish Data Every 3 Seconds -----

    If (millis() - lastPublish > 3000) {
```

```
    lastPublish = millis();  
    sendData(temp, hum, light);  
}  
  
Delay(200);  
}
```

Readings

----- GREENHOUSE DATA -----

Temperature: -35.20 °C

Humidity: 12.50 %

Light Avg: 100

Heater: 1

Fan: 0

Light: 1

----- GREENHOUSE DATA -----

Temperature: -35.20 °C

Humidity: 12.50 %

Light Avg: 1001

Heater: 1

Fan: 0

Light: 1

----- GREENHOUSE DATA -----

Temperature: 40.50 °C

Humidity: 12.50 %

Light Avg: 1001

Heater: 0

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: 40.50 °C

Humidity: 81.50 %

Light Avg: 1001

Heater: 0

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: 40.50 °C

Humidity: 81.50 %

Light Avg: 3917

Heater: 0

Fan: 1

Light: 0

----- GREENHOUSE DATA -----

Temperature: 40.50 °C

Humidity: 81.50 %

Light Avg: 57

Heater: 0

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: 40.50 °C

Humidity: 35.50 %

Light Avg: 57

Heater: 0

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: 68.10 °C

Humidity: 35.50 %

Light Avg: 57

Heater: 0

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: 68.10 °C

Humidity: 35.50 %

Light Avg: 57

Heater: 0

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: -33.10 °C

Humidity: 81.00 %

Light Avg: 57

Heater: 1

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: -33.10 °C

Humidity: 81.00 %

Light Avg: 57

Heater: 1

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: 75.90 °C

Humidity: 81.00 %

Light Avg: 57

Heater: 0

Fan: 1

Light: 1

----- GREENHOUSE DATA -----

Temperature: 75.90 °C

Humidity: 81.00 %

Light Avg: 57

Heater: 0

Fan: 1

Light: 1

•

Q46. River Water Level Alert (Flood Detection)

Problem Statement

Given the frequent flooding near the Tawi and Chenab rivers, design a low-cost IoT flood-early-warning sensor node using an HC-SR04 ultrasonic sensor to monitor water level in a model river (a glass tank). Trigger alerts at predefined levels and transmit data wirelessly.

Requirements:

- HC-SR04 mounted above water surface; distance decreasing = water rising
- 3 alert levels: NORMAL (>30cm clearance), WARNING (15–30cm), CRITICAL (<15cm)
- LED + buzzer alerts with different patterns per level
- Rolling 10-sample average to filter ultrasonic noise
- Wi-Fi/BT broadcast of alert level as a simple string message
- Web dashboard showing live level graph with Chart.js

Answer46; An **ESP32** continuously measures the distance between an **HC-SR04 ultrasonic sensor** and the water surface. Since the sensor is mounted above the water, a **decrease in measured distance indicates a rise in water level**.

To reduce fluctuations caused by ultrasonic sensor noise, the system calculates a **rolling average** of recent distance measurements before transmitting the data to the cloud.

Three flood alert levels are implemented:

-  **NORMAL** – Safe water level
-  **WARNING** – Water level rising
-  **CRITICAL** – Immediate flood risk

Each level activates a different LED and buzzer pattern. The measured water level and alert status are uploaded to **ThingSpeak** over Wi-Fi for remote monitoring and live visualization.

🔧 Hardware Components

Component	Quantity
----- :-----☺	
ESP32 Dev Module	1
HC-SR04 Ultrasonic Sensor	1
Active Buzzer	1
Green LED	1
Yellow LED	1
Red LED	1
220Ω Resistors	3
Jumper Wires	As required

🔌 Pin Connections

Component	ESP32 Pin		
<table border="1"><tr><td></td><td></td></tr></table>			
HC-SR04 TRIG	GPIO5		
HC-SR04 ECHO	GPIO18		
Active Buzzer	GPIO4		
Green LED	GPIO12		

| Yellow LED | GPIO13 |

| Red LED | GPIO14 |

Flood Alert Levels

| Water Clearance | Alert Level |

--	--

| **>30 cm** | □ NORMAL |

| **15–30 cm** | □ WARNING |

| **<15 cm** | ● CRITICAL |

Since the ultrasonic sensor is mounted above the river:

- Higher distance → Lower water level
 - Lower distance → Higher water level
-

System Features

- Continuous water level monitoring
- HC-SR04 ultrasonic sensing
- Three-stage flood warning system

- LED and buzzer alerts
 - Rolling average filtering to reduce sensor noise
 - Wi-Fi cloud connectivity
 - ThingSpeak data logging
 - Real-time Serial Monitor output
-

Cloud Dashboard

Sensor readings are uploaded to **ThingSpeak** using HTTP requests.

Uploaded Fields

Field	Description
-------	-------------

--	--

Field 1	Average Water Distance (cm)
---------	-----------------------------

Field 2	Flood Alert Level
---------	-------------------

ThingSpeak automatically generates live graphs for monitoring water level trends remotely.

Alert Behaviour

□ NORMAL

- Green LED ON
- Buzzer OFF

□ WARNING

- Yellow LED ON
- Short intermittent buzzer

● CRITICAL

- Red LED ON
 - Continuous high-frequency buzzer
-

📄 Sample Serial Output

```text

Raw: 41.2 cm | Avg: 40.6 cm

Level: NORMAL

Raw: 23.5 cm | Avg: 24.1 cm

Level: WARNING

Raw: 10.7 cm | Avg: 11.4 cm

Level: CRITICAL

Sending to ThingSpeak...

HTTP Response: 200

...

---

#### Libraries Used

- WiFi.h
- HTTPClient.h

---

#### Simulation Notes

This project was developed using **\*\*Wokwi\*\***.

- The HC-SR04 sensor simulates river water level measurements.
- LEDs are used to represent different flood alert stages.
- ThingSpeak is used as the cloud dashboard to visualize sensor readings in real time.



- The buzzer patterns simulate emergency warning levels for nearby residents.



## **Code**

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
Const char* ssid = "Wokwi-GUEST";
```

```
Const char* password = "";
```

```
// ----- THINGSPEAK -----
```

```
String apiKey = "APIKEY";
```

```
Const char* server = http://api.thingspeak.com/update;
```

```
#define TRIG_PIN 5
```

```
#define ECHO_PIN 18
```

```
#define BUZZER 4
```

```
#define LED_GREEN 12
```

```
#define LED_YELLOW 13
```

```
#define LED_RED 14
```

```
Float readings[5];
```

```
Int indexNum = 0;
```

```
Void setup_wifi() {
```

```
 Serial.print("Connecting WiFi...");
```

```
 WiFi.begin(ssid, password);
```

```
 While (WiFi.status() != WL_CONNECTED) {
```

```
 Delay(500);
```

```
 Serial.print(".");
```

```
 }
```

```
 Serial.println("\nWiFi Connected!");
```

```
}
```

```
Float getDistance() {
 digitalWrite(TRIG_PIN, LOW);
 delayMicroseconds(2);

 digitalWrite(TRIG_PIN, HIGH);
 delayMicroseconds(10);
 digitalWrite(TRIG_PIN, LOW);

 long duration = pulseIn(ECHO_PIN, HIGH);
 float distance = duration * 0.034 / 2;

 return distance;
}
```

```
// ----- AVERAGE -----
```

```
Float getAverage(float val) {
 Readings[indexNum] = val;
 indexNum = (indexNum + 1) % 5;

 float sum = 0;
 for (int I = 0; I < 5; i++) {
 sum += readings[i];
 }
```

```
Return sum / 5;

}

Void sendToThingSpeak(float distance, int levelValue) {

 If (WiFi.status() == WL_CONNECTED) {

 HTTPClient http;

 String url = String(server) + "?api_key=" + apiKey +
 "&field1=" + String(distance) +
 "&field2=" + String(levelValue);

 Serial.println("Sending to TS:");
 Serial.println(url); // DEBUG

 http.begin(url);

 int httpCode = http.GET();

 Serial.print("HTTP Response: ");
 Serial.println(httpCode);

 http.end();
 }
}
```

```
}
```

```
Void setup() {
```

```
 Serial.begin(115200);
```

```
 pinMode(TRIG_PIN, OUTPUT);
```

```
 pinMode(ECHO_PIN, INPUT);
```

```
 pinMode(BUZZER, OUTPUT);
```

```
 pinMode(LED_GREEN, OUTPUT);
```

```
 pinMode(LED_YELLOW, OUTPUT);
```

```
 pinMode(LED_RED, OUTPUT);
```

```
 for (int I = 0; I < 5; i++) readings[i] = 30;
```

```
 setup_wifi();
```

```
 // Buzzer test
```

```
 Tone(BUZZER, 1000);
```

```
 Delay(300);
```

```
 noTone(BUZZER);
```

```
}
```

```
Void loop() {
```

```
Float rawDistance = getDistance();
```

```
Float avgDistance = getAverage(rawDistance);
```

```
Serial.print("Raw: ");
```

```
Serial.print(rawDistance);
```

```
Serial.print(" cm | Avg: ");
```

```
Serial.println(avgDistance);
```

```
String level;
```

```
Int levelValue;
```

```
digitalWrite(LED_GREEN, LOW);
```

```
digitalWrite(LED_YELLOW, LOW);
```

```
digitalWrite(LED_RED, LOW);
```

```
noTone(BUZZER);
```

```
// 🚫 CRITICAL (<15 cm) → instant
```

```
If (rawDistance < 15) {
```

```
 Level = "CRITICAL";
```

```
 levelValue = 2;
```

```
 digitalWrite(LED_RED, HIGH);
```

```
tone(BUZZER, 3000);
}

Else if (rawDistance >= 15 && rawDistance <= 30) {
 Level = "WARNING";
 levelValue = 1;

 digitalWrite(LED_YELLOW, HIGH);

 tone(BUZZER, 2000);
 delay(200);
 noTone(BUZZER);
}

Else {
 Level = "NORMAL";
 levelValue = 0;

 digitalWrite(LED_GREEN, HIGH);
}

Serial.println("Level: " + level);

// 📡 Send to ThingSpeak
```

```
sendToThingSpeak(avgDistance, levelValue);

// ⚠ IMPORTANT: ThingSpeak limit
Delay(15000);
}
```

---

## Q47. Heart Rate & SpO2 Monitor (Simulation)

### Problem Statement

Using a MAX30102 pulse oximeter module (or simulate with an LDR as a proxy sensor),

build a health monitoring prototype that measures pulse rate and blood oxygen saturation,

displays values, and alerts on abnormal readings. This is a simulation for learning sensor

interfacing, not a medical device.

### Requirements:

- MAX30102 via I2C (or LDR analog simulation with smoothed readings)
- Display: LCD or Serial Monitor with readings updated every 2 seconds
- Alert: Red LED if HR < 50 or > 120 bpm, or SpO2 < 94%



- Implement a 5-reading rolling average to smooth noisy data
- Log readings to a /data/health\_log.csv file reference in GitHub README

## Answer47:Problem Statement

Design a health monitoring prototype using a MAX30102 pulse oximeter (or simulate using an LDR). The system measures Heart Rate (HR) and Blood Oxygen Saturation (SpO<sub>2</sub>), displays values every 2 seconds, alerts on abnormal readings, and stores the readings in a CSV log.

---

## Components Required

- ESP32
- MAX30102 Sensor (or LDR for simulation)
- 16×2 LCD (optional)
- Red LED
- Breadboard & Jumper Wires

---

## Circuit Connections

| Component    | ESP32 Pin |
|--------------|-----------|
| MAX30102 SDA | GPIO21    |
| MAX30102 SCL | GPIO22    |
| Red LED      | GPIO25    |
| LCD SDA      | GPIO21    |
| LCD SCL      | GPIO22    |

---

## Working Algorithm

1. Read Heart Rate and SpO<sub>2</sub> values every 2 seconds.
2. Calculate the rolling average of the last five readings.
3. Display HR and SpO<sub>2</sub> on the LCD/Serial Monitor.
4. Turn on the Red LED if:
  - HR < 50 bpm or HR > 120 bpm
  - SpO<sub>2</sub> < 94%
5. Save readings to health\_log.csv.

```
#include <Arduino.h>
```

```
#define LDR_PIN 34

#define LED_PIN 14

const int numReadings = 5;

int readings[numReadings];

int indexR = 0;

int total = 0;

int average = 0;

void setup() {

 Serial.begin(115200);

 pinMode(LED_PIN, OUTPUT);

 for (int i = 0; i < numReadings; i++) readings[i] = 0;

}

void loop() {

 int sensorValue = analogRead(LDR_PIN);

 // smoothing

 total -= readings[indexR];

 readings[indexR] = sensorValue;

 total += readings[indexR];
```

```
indexR = (indexR + 1) % numReadings;

average = total / numReadings;

// WIDE mapping (fix)

int heartRate = map(average, 0, 4095, 40, 140);

int spo2 = map(average, 0, 4095, 88, 100);

// ALERT logic (unchanged)

bool alert = (heartRate < 50 || heartRate > 120 || spo2 < 94);

digitalWrite(LED_PIN, alert ? HIGH : LOW);

// DEBUG

Serial.println("-----");

Serial.print("Heart Rate : ");

Serial.print(heartRate);

Serial.println(" bpm");

Serial.print("SpO2 : ");

Serial.print(spo2);

Serial.println(" %");

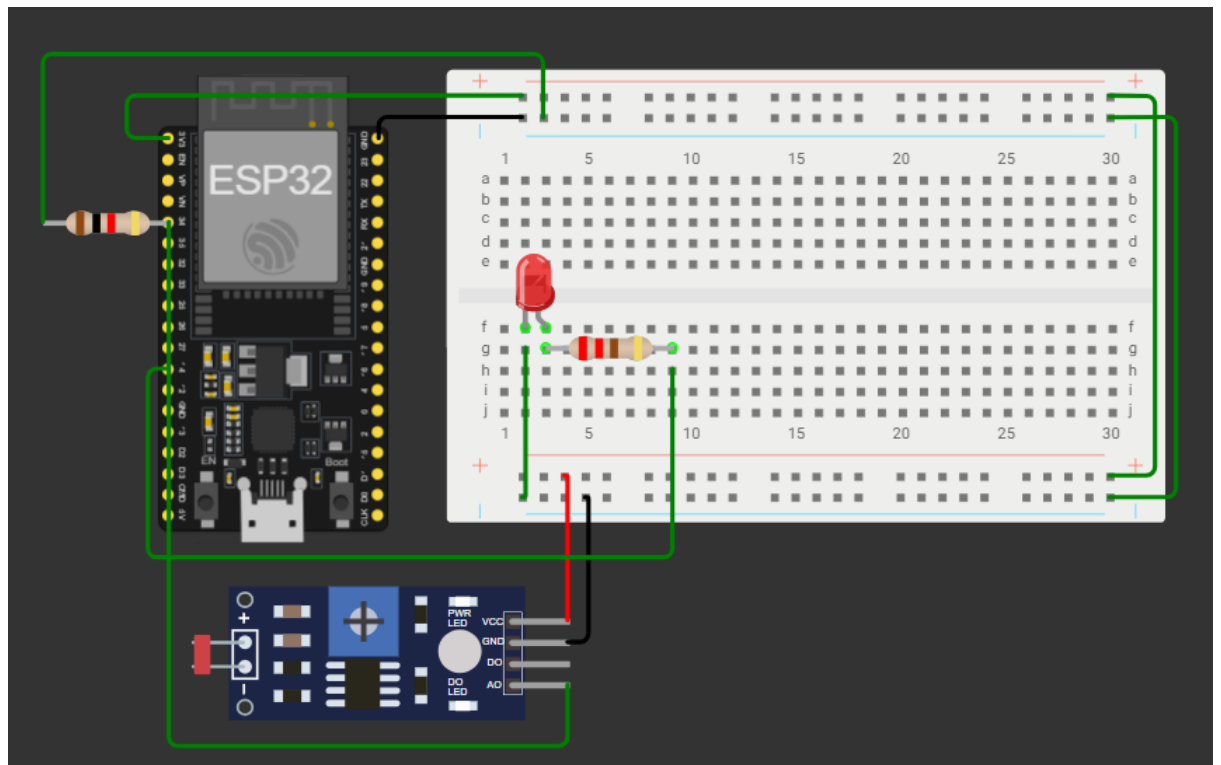
Serial.print("Alert : ");

Serial.println(alert ? "YES" : "NO");

Serial.println("-----");
```

```
delay(2000);
```

```
}
```



|                     |
|---------------------|
| Heart Rate : 44 bpm |
| SpO2 : 88 %         |
| Alert : YES         |
| -----               |
| -----               |
| Heart Rate : 49 bpm |
| SpO2 : 89 %         |
| Alert : YES         |
| -----               |
| -----               |
| Heart Rate : 54 bpm |
| SpO2 : 89 %         |
| Alert : YES         |
| -----               |
| -----               |
| Heart Rate : 59 bpm |
| SpO2 : 90 %         |
| Alert : YES         |
| -----               |
| -----               |
| Heart Rate : 64 bpm |
| SpO2 : 90 %         |
| Alert : YES         |
| -----               |
| -----               |
| Heart Rate : 64 bpm |
| SpO2 : 90 %         |
| Alert : YES         |
| -----               |
| -----               |
| Heart Rate : 64 bpm |
| SpO2 : 90 %         |
| Alert : YES         |

#### 48. Design Challenge: IoT for Jammu Smart City

J&K Government is planning a Smart City project for Jammu. As an IoT engineer, propose a system to monitor and manage 3 of the following (choose any 3): Street lighting, Traffic flow, Parking availability, Air quality, Water supply pressure, Waste bin fill levels. For each:

- Identify the sensor(s) needed

- Identify the microcontroller / connectivity module
- Describe the data flow: Sensor → Protocol → Cloud → Dashboard
- Estimate approximate cost per node (in INR)

*Present your answer as a structured proposal (500–700 words) uploaded as /final\_project/smart\_city\_proposal.md in GitHub.*

*1. Write the code for each problem separately*

**Answer48: Code for street light**

```
#define LDR_PIN 34

#define PIR_PIN 27

#define LED_PIN 25

void setup() {

 Serial.begin(115200);

 pinMode(PIR_PIN, INPUT);
 pinMode(LED_PIN, OUTPUT);

 Serial.println("Smart Street Lighting
System Started");
```

```
}
```

```
void loop() {
```

```
 int lightValue = analogRead(LDR_PIN);
```

```
 bool motion = digitalRead(PIR_PIN);
```

```
 bool streetLight = false;
```

```
 // Night condition
```

```
 if (lightValue < 1800) {
```

```
 if (motion) {
```

```
 streetLight = true;
```

```
 }
```

```
 }
```

```
digitalWrite(LED_PIN, streetLight);
```

```
Serial.println("-----");
```

```
Serial.print("LDR Value : ");
```

```
Serial.println(lightValue);
```

```
Serial.print("Motion : ");
```

```
Serial.println(motion ? "Detected" : "Not
Detected");
```

```
Serial.print("Street Light : ");
```

```
Serial.println(streetLight ? "ON" : "OFF");
```

```
if (lightValue >= 1800)
```

```
 Serial.println("Status : Daytime");
```



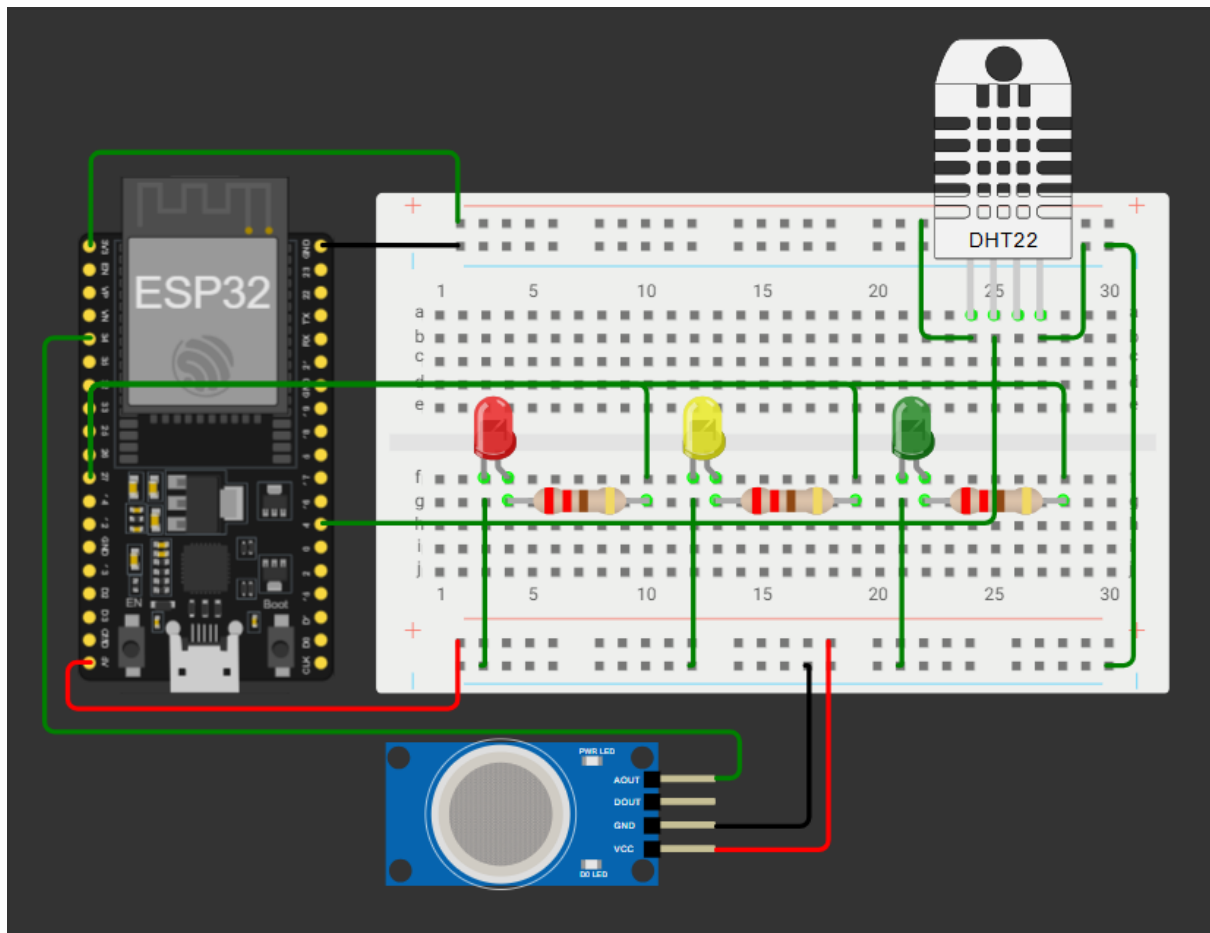
else

```
Serial.println("Status : Night");
```

```
delay(1000);
```

```
}
```

## Circuit Diagram



Code for Smart Waste bin:

```
#define TRIG_PIN 5
```

```
#define ECHO_PIN 18
```

```
#define GREEN_LED 25
```

```
#define YELLOW_LED 26
```

```
#define RED_LED 27
```

```
float getDistance() {
```

```
 digitalWrite(TRIG_PIN, LOW);
```

```
 delayMicroseconds(2);
```

```
 digitalWrite(TRIG_PIN, HIGH);
```

```
 delayMicroseconds(10);
```

```
 digitalWrite(TRIG_PIN, LOW);
```

```
 long duration = pulseIn(ECHO_PIN, HIGH);
```

```
 float distance = duration * 0.034 / 2;
```

```
 return distance;
```

```
}
```

```
void setup() {
```

```
Serial.begin(115200);
```

```
pinMode(TRIG_PIN, OUTPUT);
```

```
pinMode(ECHO_PIN, INPUT);
```

```
pinMode(GREEN_LED, OUTPUT);
```

```
pinMode(YELLOW_LED, OUTPUT);
```

```
pinMode(RED_LED, OUTPUT);
```

```
Serial.println("Smart Waste Bin Monitoring");
```

```
}
```

```
void loop() {
```

```
float distance = getDistance();
```

```
digitalWrite(GREEN_LED, LOW);
```

```
digitalWrite(YELLOW_LED, LOW);
```

```
digitalWrite(RED_LED, LOW);
```

```
String status;
```

```
// Bin nearly empty
```

```
if (distance > 20) {
```

```
 status = "EMPTY";
```

```
 digitalWrite(GREEN_LED, HIGH);
```

```
}
```

```
// Bin half full
```

```
else if (distance > 10) {
```

```
 status = "HALF FULL";
```

```
 digitalWrite(YELLOW_LED, HIGH);
```

```
}
```

```
// Bin full
```

```
else {
```

```
 status = "FULL";
```

```
 digitalWrite(RED_LED, HIGH);
```

```
}
```

```
Serial.println("-----");
```

```
Serial.print("Distance : ");
```

```
Serial.print(distance);
```

```
Serial.println(" cm");
```

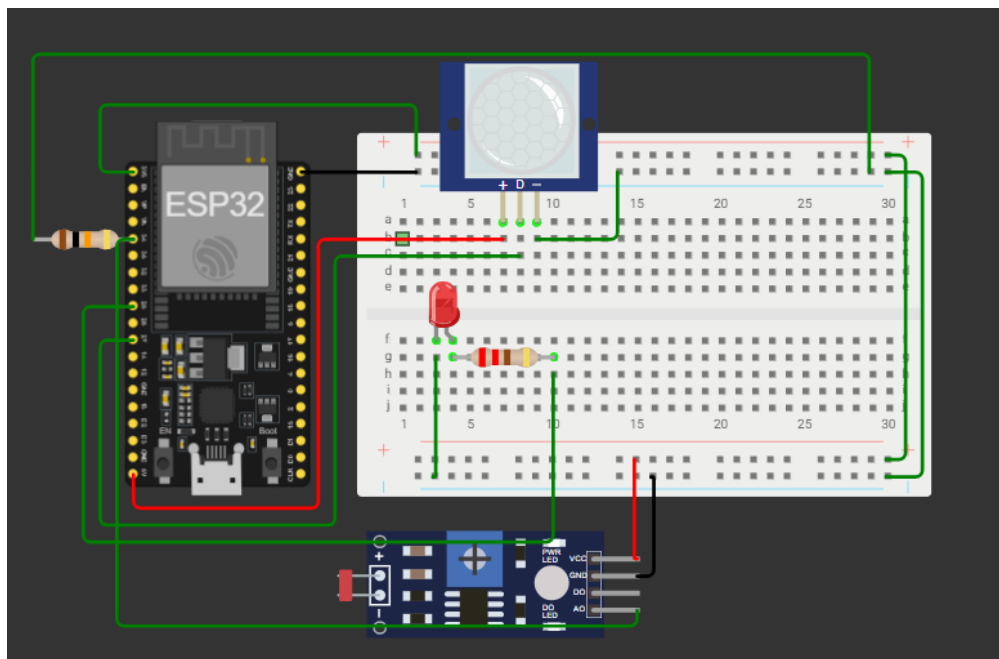
```
Serial.print("Bin Status : ");
```

```
Serial.println(status);
```

```
delay(2000);
```

```
}
```

**Circuit diagram for smart waste bin:**



**Code for AQI MOITORING**

```
#include <DHT.h>
```

```
#define DHTPIN 4
```

```
#define DHTTYPE DHT22
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
#define MQ2_PIN 34
```

```
#define RED_PIN 25
```

```
#define GREEN_PIN 26
```

```
#define BLUE_PIN 27
```

```
#define BUZZER 14
```

```
int baseline = 0;
```

```
void setColor(bool r, bool g, bool b) {
```

```
 digitalWrite(RED_PIN, r);
```

```
 digitalWrite(GREEN_PIN, g);
```

```
 digitalWrite(BLUE_PIN, b);
```

```
}
```

```
void setup() {
```

```
 Serial.begin(115200);
```

```
dht.begin();
```

```
pinMode(RED_PIN, OUTPUT);
```

```
pinMode(GREEN_PIN, OUTPUT);
```

```
pinMode(BLUE_PIN, OUTPUT);
```

```
pinMode(BUZZER, OUTPUT);
```

```
Serial.println("Stabilizing MQ-2... (10 sec)");
```

```
delay(10000);
```

```
int sum = 0;
```

```
for (int i = 0; i < 30; i++) {
```

```
 sum += analogRead(MQ2_PIN);
```

```
 delay(200);
```

```
}
```

```
baseline = sum / 30;
```

```
Serial.print("Baseline: ");
```

```
Serial.println(baseline);
```

```
Serial.println("Timestamp, MQ-2 Value, DHT11 Temp, DHT11 Humidity,
Severity Level");
```

```
}
```

```
void loop() {
```

```
int gasRaw = analogRead(MQ2_PIN);
```

```
int diff = gasRaw - baseline;
```

```
float temp = dht.readTemperature();
```

```
float hum = dht.readHumidity();
```

```
if (isnan(temp) || isnan(hum)) {
```

```
 Serial.println("DHT ERROR");
```

```
 delay(2000);
```

```
 return;
```

```
}
```

```
String status;
```

```
// THRESHOLDS
```

```
if (diff < 80) {
```

```
 status = "SAFE";
```

```
 setColor(LOW, HIGH, LOW); // GREEN
```

```
 noTone(BUZZER);
```

```
}
```

```
else if (diff < 250) {
```

```
 status = "MODERATE";
```



```
setColor(HIGH, HIGH, LOW); // YELLOW

tone(BUZZER, 1000);

delay(200);

noTone(BUZZER);
}

else {

 status = "DANGER";

 setColor(HIGH, LOW, LOW); // RED

 tone(BUZZER, 2000);
}

// TIME

int seconds = millis() / 1000;

int mins = seconds / 60;

int secs = seconds % 60;

char timeStr[6];

sprintf(timeStr, "%02d:%02d", mins, secs);

// CSV

Serial.print(timeStr);
```

```
Serial.print(", ");

Serial.print(gasRaw);

Serial.print(" ppm, ");

Serial.print(temp);

Serial.print("°C, ");

Serial.print(hum);

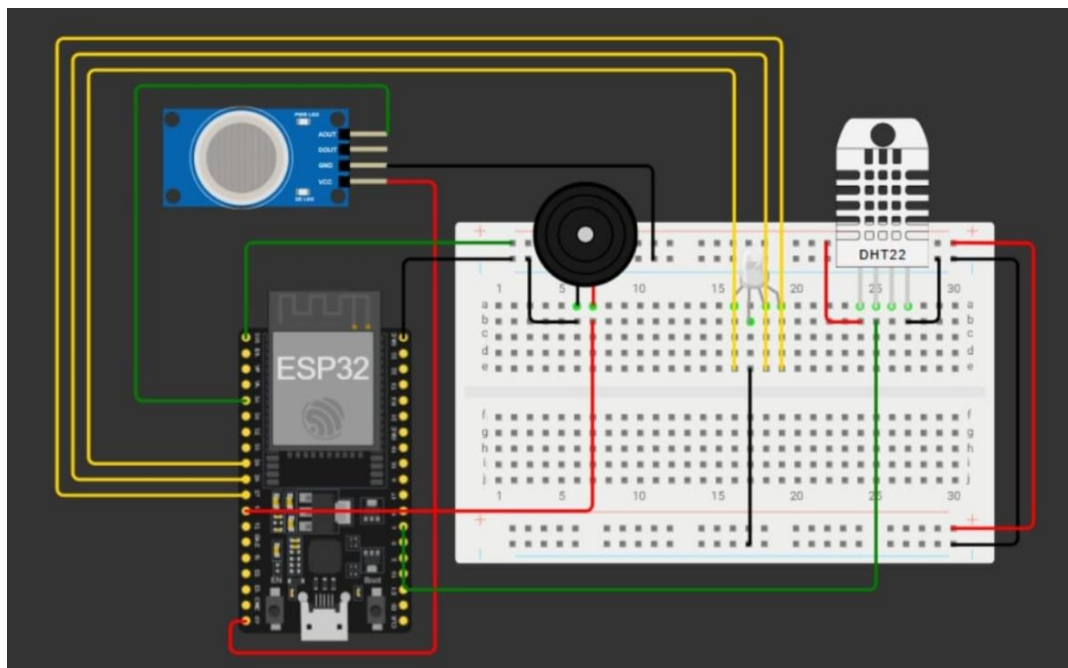
Serial.print("%, ");

Serial.println(status);

delay(2000);

}
```

### Circuit diagram for AQI Monitoring



#### **Q49. Security Vulnerability Analysis**

You are given a poorly written IoT code snippet that connects an ESP8266 to a public Wi-Fi

network. The code hardcodes the SSID and password, uses unencrypted HTTP (not HTTPS),

publishes sensor data to a public MQTT broker without authentication, and never validates

incoming commands. Identify at least 5 security vulnerabilities, explain the risk of each, and

provide corrected code for at least 2

#### **Answer49:1. Hardcoded Wi-Fi Credentials**

**Risk:** Anyone with access to the source code can obtain the Wi-Fi password and compromise the network.

**Solution:** Store credentials in a separate configuration file (`config.h`) and add it to `.gitignore`.

---

#### **2. Using HTTP Instead of HTTPS**

**Risk:** Data is transmitted without encryption, allowing attackers to intercept or modify messages.

**Solution:** Use HTTPS with SSL/TLS encryption.

---

#### **3. Public MQTT Broker Without Authentication**

**Risk:** Anyone can publish or subscribe to sensor data.

**Solution:** Use username/password authentication and a private MQTT broker.

---

#### 4. No Validation of Incoming Commands

**Risk:** Attackers can send malicious commands to control the IoT device.

**Solution:** Validate all incoming commands and allow only trusted inputs.

---

#### 5. No Firmware Updates

**Risk:** Known vulnerabilities remain unpatched, increasing the risk of exploitation.

**Solution:** Implement secure OTA (Over-the-Air) firmware updates.

---

## Corrected Code Example

### Secure Wi-Fi Credentials

```
#include "config.h"

const char* ssid = WIFI_SSID;
const char* password = WIFI_PASSWORD;
```

### Secure HTTPS Connection

```
WiFiClientSecure client;

client.setCACert(rootCACertificate);

HTTPClient https;

https.begin(client, "https://example.com/api");
```

---

## Q50. Reflection & Learning Portfolio

Describe what you built and what you learned.  
How will you apply IoT skills in your field of study

or future career? What would you build next — describe your own IoT project idea with sensors,

connectivity, and application. (Minimum 400 words, personal and reflective)

**Answer50:** This Internet of Things (IoT) course has greatly improved my understanding of smart devices, embedded systems, and wireless communication. Throughout the assignment, I learned how sensors, microcontrollers, cloud services, and communication protocols work together to build intelligent systems that can monitor and control real-world environments.

One of the most interesting parts of the course was working with ESP32 and ESP8266 microcontrollers. I learned how to interface different sensors such as DHT11, MQ-2 gas sensors, ultrasonic sensors, PIR motion sensors, LDRs, and soil moisture sensors. I also understood how these sensors collect environmental data and how the microcontroller processes that information to make automatic decisions.

Another important lesson was learning different communication technologies such as Wi-Fi, Bluetooth, MQTT, HTTP, and REST APIs. I now understand why MQTT is widely used in IoT because of its low power consumption and efficient publish-subscribe communication model. I also learned the importance of securing IoT devices using HTTPS, WPA3, certificate pinning, and strong authentication methods.

This assignment also improved my programming skills in Arduino IDE. I gained experience in writing modular programs, using conditional statements, timers with `millis()`, sensor calibration, and controlling actuators such as relays, LEDs, buzzers, and servo motors. Designing complete IoT systems from problem statements helped me think like an engineer by considering reliability, scalability, and user safety.

In my future career, I hope to use IoT technology in agriculture and smart city applications. Smart irrigation systems, flood monitoring systems, and environmental monitoring solutions can significantly improve productivity and public safety in Jammu and Kashmir. These technologies can help farmers save water, reduce crop losses, and improve decision-making through real-time monitoring.

If I were to build my own IoT project next, I would develop a **Smart Landslide and Flood Monitoring System** for hilly regions of Jammu and Kashmir. The system would use **rainfall sensors, soil moisture sensors, ultrasonic water level sensors, GPS modules, and ESP32 controllers**. Data would be transmitted using **LoRaWAN** for long-distance communication to a cloud platform. A mobile application would display real-time alerts and warning messages to local authorities and residents. This system could help reduce disaster risks and improve emergency response during heavy rainfall and flooding.

Overall, this course has provided me with practical knowledge of IoT hardware, software, networking, cloud integration, and cybersecurity. It has strengthened my confidence in designing intelligent embedded systems and inspired me to continue exploring advanced IoT technologies for solving real-world problems.